Book Management Backend — Complete build Guide

> **A progressive, hands-on tutorial** to build a production-ready TypeScript backend from scratch. This guide teaches you modern backend development through building a real-world library management system.

---

1. About This Guide

🎯 Who Is This For?

This guide is for **students and developers** who want to learn professional backend development with TypeScript, Express, and MongoDB (mongoose).

You should be comfortable with:

- JavaScript fundamentals (variables, functions, promises, async/await)
- Basic Node.js concepts (npm, modules, packages)
- JSON and REST API concepts
- Command line basics

📚 How to Use This Guide

This is a hands-on tutorial, not a reference manual. You should:

1. **Follow in order:** Each section builds on previous ones
2. **Type the code yourself:** Don't just copy-paste
3. **Read the explanations:** Understand the "why" behind each decision
4. **Test as you go:** Use Postman or Thunder Client after each feature
5. **Experiment:** Try breaking things to understand how they work

> **Important**: Don't skip ahead. The complexity increases gradually, and later sections assume you understand earlier concepts.

📖 Reading Order

Don't read everything at once. This guide is organized into **4 sprints**. Complete each sprint before moving to the next:

1. **Sprint 1**: Get a working server with authentication and user management
2. **Sprint 2**: Add the core domain — books, authors, categories, ratings
3. **Sprint 3**: Build the borrow system with complex business logic
4. **Sprint 4**: Add orders with transactions and the admin dashboard

The **Cross-Cutting Concerns** can be read alongside any sprint — they contain best practices that apply throughout.

---

2. What You'll Build

You'll create a **complete library management system** with these features:

Core Features

User Management:

- User registration and authentication
- Profile management
- Admin and regular user roles

Book Catalog:

- Complete book inventory with authors and categories
- Advanced search
- Book ratings and reviews
- Stock management

Borrowing System:

- Request and approve book loans
- Track due dates and returns
- Automatic overdue detection
- Late fee calculation
- Borrow history

Order Management:

- Purchase books online
- Shopping cart functionality
- Order status tracking
- Transaction safety with MongoDB transactions

Admin Dashboard:

- Real-time statistics
- User and inventory analytics
- Revenue tracking
- Low stock alerts

Technical Features

Security:

- Password hashing with bcrypt
- JWT-based authentication
- Role-based access control
- Input validation
- Security headers

Performance:

- Database indexing
- Pagination
- Efficient queries

Code Quality:

- TypeScript strict mode
- Consistent error handling
- Comprehensive validation

---

3. Technology Stack

Core Technologies

Technology	Version	Purpose	Why This Choice
Node.js	20+	JavaScript runtime	Modern, async I/O, large ecosystem
TypeScript	5.x	Static typing	Catch errors at compile time, better IDE support
Express	5.x	Web framework	Most popular, flexible, middleware-based
MongoDB	6+	Database	Document model fits JSON APIs, flexible schema
Mongoose	8.x	ODM	Schema validation, middleware, type safety

Key Libraries

Package	Purpose	Why Important
Zod	Runtime validation	Type-safe request validation with TypeScript inference
bcrypt	Password hashing	Industry standard, adaptive cost factor
jsonwebtoken	JWT tokens	Stateless authentication, widely supported
helmet	Security headers	Protects against common web vulnerabilities
cors	Cross-origin requests	Enable frontend communication
http-status-codes	HTTP constants	Named constants instead of magic numbers
morgan	Request logging	Development debugging

Development Tools

- **tsx** — Fast TypeScript execution with watch mode
- **ESLint** — Code quality and consistency
- **typescript-eslint** — TypeScript-specific linting rules

---

4. Architecture Overview

Layered Architecture

Our application follows a **clean, layered architecture** with clear separation of concerns:

```
┌──────────────────────────────────────┐
│          HTTP Request                │
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│        Router Layer                  │  ← Maps URLs to handlers
│  • Route definitions                 │
│  • Middleware chains                 │
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│      Middleware Layer                │  ← Cross-cutting concerns
│  • Authentication (JWT verify)       │
│  • Validation (Zod schemas)          │
│  • Error handling                    │
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│       Handler Layer                  │  ← Business logic
│  • Request processing                │
│  • Response formatting               │
│  • Error throwing                    │
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│        Model Layer                   │  ← Data access
│  • Mongoose schemas                  │
│  • Database queries                  │
│  • Validations & hooks               │
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│         MongoDB                      │
└──────────────────────────────────────┘
```

Supporting Layers

Types: TypeScript interfaces and type definitions

- Define data shapes
- Enable type inference
- Catch type errors early

Validation: Zod schemas for runtime validation

- Validate request payloads
- Transform and sanitize data
- Generate TypeScript types

Utils: Shared helper functions

- Date calculations
- Business logic helpers
- Response formatting
- Logging

Services: External integrations

- Database connection

Code Organization Philosophy

File Naming:

- Singular for domain concepts (`user.ts` , not `users.ts`)
- Descriptive suffixes (`userSchema` , `userModel` , `userRouter`)
- Consistent across layers

Function Naming:

- Verb-first for actions (`getUsers` , `createBook`)
- Descriptive and specific (`getUserById` , not `get`)
- Matches HTTP method intent

Variable Naming:

- `camelCase` for functions and variables
- `PascalCase` for types and classes
- `UPPER_SNAKE_CASE` for constants

---

5. Project Initialization & Configuration

5.1 Create the project directory

```
# Create and enter the project directory
mkdir book-management-back
cd book-management-back

# Initialize npm with default values
npm init -y
```

This creates a `package.json` file with default values.

5.2 Configure package.json

Fields to set:

- `name` : `"book-management-back"`
- `version` : `"1.0.0"`
- `description` : `"Book management system backend with TypeScript, Express, and MongoDB"`
- `main` : `"dist/index.js"` — Points to the compiled JavaScript output
- `type` : `"module"` — Enables native ES module syntax (import/export), which requires `.js` extensions in import paths even for TypeScript files

Scripts to define:

```
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "typecheck": "tsc --noEmit",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix"
  }
}
```

Script	Purpose
`dev`	Development server with hot reload via tsx
`build`	Compile TypeScript to JavaScript
`start`	Run the production build
`typecheck`	Type-check without generating output files
`lint` / `lint:fix`	Run/fix linting issues

5.3 Install Dependencies

Install **production dependencies**:

```
npm install express dotenv cors helmet mongoose bcrypt jsonwebtoken zod http-status-codes
```

Package	Purpose
`express`	Web framework for routing and middleware
`dotenv`	Load environment variables from `.env` file
`cors`	Enable Cross-Origin Resource Sharing
`helmet`	Set security-related HTTP headers
`mongoose`	MongoDB object modeling (ODM)
`bcrypt`	Password hashing with salt
`jsonwebtoken`	Create and verify JWT tokens
`zod`	Schema validation with TypeScript inference
`http-status-codes`	Named HTTP status constants (200, 404, etc.)

Development:

```
npm install -D typescript tsx @types/node @types/express @types/express-serve-static-core @types/bcrypt @types/jsonwebtoken @types/cors
```

Package	Purpose
`typescript`	TypeScript compiler
`tsx`	Fast TypeScript executor with watch mode
`@types/*`	TypeScript type definitions for JavaScript libraries
`eslint`	Code linting for quality and consistency
`@eslint/js`	ESLint JavaScript configurations
`typescript-eslint`	ESLint TypeScript support

---

6. TypeScript & Linting Configuration

6.1 Create tsconfig.json

Create a file named `tsconfig.json` in the project root:

```
{
  "compilerOptions": {
    // === Output Configuration ===
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",

    // === Type Checking ===
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,

    // === Module Resolution ===
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,

    // === Emit ===
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,

    // === Advanced ===
    "useDefineForClassFields": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

6.3 Configuration Explained

Output Configuration:

- `target: "ES2022"` — Compile to modern JavaScript (Node 20+ supports this)
- `module: "NodeNext"` — Use ES modules with `.js` extensions
- `outDir: "./dist"` — Compiled JavaScript goes here
- `rootDir: "./src"` — Source TypeScript files live here

Strict Type Checking:

- `strict: true` — Enables all strict type checking options
- `noUnusedLocals: true` — Error on unused variables
- `noUnusedParameters: true` — Error on unused function parameters
- `noImplicitReturns: true` — All code paths must return a value
- `noFallthroughCasesInSwitch: true` — Prevent accidental switch fallthrough

> 💡 **Why strict mode from the start?**
> Strict mode catches errors early. It's harder to enable later when you already have code with type issues. Starting strict teaches good habits.

Sometimes you need to accept parameters you don't use (like Express middleware):

```
// ❌ This will error with noUnusedParameters
function middleware(req, res, next) {
  // Only using req
  console.log(req.body);
}

// ✅ Prefix unused params with underscore
function middleware(req, _res, _next) {
  console.log(req.body);
}
```

TypeScript ignores parameters starting with `_`.

6.4 ESLint Configuration

Create `eslint.config.js` in the project root:

```
import eslint from "@eslint/js";
import tseslint from "typescript-eslint";

export default [
  // Apply recommended rules
  eslint.configs.recommended,
  ...tseslint.configs.recommended,

  // Ignore compiled files
  {
    ignores: ["dist/", "node_modules/"],
  },

  // Custom rules
  {
    rules: {
      // Allow unused vars if prefixed with underscore
      "@typescript-eslint/no-unused-vars": [
        "error",
        {
          argsIgnorePattern: "^_",
          varsIgnorePattern: "^_",
        },
      ],

      // Let TypeScript infer return types
      "@typescript-eslint/explicit-function-return-type": "off",

      // Warn but don't block on 'any' usage
      "@typescript-eslint/no-explicit-any": "warn",

      // Require semicolons
      semi: ["error", "always"],

      // Require double quotes
      quotes: ["error", "double"],
    },
  },
];
```

---

7. Environment Variables

7.1 Create `.env.example`

Create `.env.example` as a template:

```
# ============================================
# Server Configuration
# ============================================
PORT=3000                                    # Port the server listens on
NODE_ENV=development                         # 'development' or 'production'

# ============================================
# Database Configuration
# ============================================
MONGODB_URI=mongodb://localhost:27017         # MongoDB connection string
MONGODB_DB_NAME=book_management              # Database name
MONGODB_USERNAME=                            # Optional: DB auth username
MONGODB_PASSWORD=                            # Optional: DB auth password

# ============================================
# Authentication (JWT)
# ============================================
AUTH_SECRET=your-super-secret-key-min-32-chars  # Secret for signing JWTs (npx auth secret)
JWT_EXPIRES_IN=7d                               # Token lifetime (e.g., '1h', '7d', '30d')

# ============================================
# CORS
# ============================================
FRONTEND_DOMAIN=http://localhost:5173         # Allowed origin for CORS
```

7.2 Create .env

- Copy the example to create your actual `.env` file

⚠️ SECURITY WARNING:

- Never commit `.env` to Git
- Change `AUTH_SECRET` to a strong random string in production using

```
npx auth secret
```

- create `.gitignore` to prevent committing sensitive files

```
# Dependencies
node_modules/

# Environment variables
.env

# Build output
dist/

# Logs
*.log
npm-debug.log*

# OS files
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/
*.swp
*.swo

# Test coverage
coverage/
```

7.3 Environment Variables Reference

Variable	Required	Default	Description
`PORT`	No	`3000`	Server port
`NODE_ENV`	No	`development`	Environment mode
`MONGODB_URI`	**Yes**	—	MongoDB connection string
`MONGODB_DB_NAME`	**Yes**	—	Database name
`MONGODB_USERNAME`	No	—	DB authentication username
`MONGODB_PASSWORD`	No	—	DB authentication password
`AUTH_SECRET`	**Yes**	—	JWT signing secret
`JWT_EXPIRES_IN`	No	`7d`	JWT token lifetime
`FRONTEND_DOMAIN`	**Yes**	—	CORS allowed origin

---

8. Project Structure

8.1 Create Folder Structure

Create the basic folder structure:

```
book-management-back/
├── node_modules/          # Dependencies (auto-generated)
├── src/
│   ├── types/             # TypeScript type definitions
│   ├── utils/             # Helper functions
│   ├── services/          # External services (DB, email, etc.)
│   ├── models/            # Mongoose schemas
│   ├── validation/        # Zod validation schemas
│   ├── middlewares/       # Express middleware
│   ├── handlers/          # Request handlers (business logic)
│   └── routers/           # Route definitions
├── .env                   # Environment variables (not in git)
├── .env.example           # Environment template (in git)
├── .gitignore             # Git ignore rules
├── eslint.config.js       # ESLint configuration
├── package.json           # Project metadata and dependencies
├── package-lock.json      # Locked dependency versions
└── tsconfig.json          # TypeScript configuration
```

8.2 When to Split Files

Split a file when **any** of these apply:

- **It exceeds ~200 lines** — Hard to scroll and find things
- **It contains unrelated concepts** — User types and Order types shouldn't live together
- **Multiple developers** would edit it simultaneously
- **Imports become circular** — A sign that responsibilities are tangled

---

9. Type System Foundation

9.1 Understanding TypeScript Types

Before writing code, understand type concepts:

What are types?

- Describe the shape of data
- Caught at compile time (before running)
- Self-documenting code
- Better IDE autocomplete

Type categories we'll create:

Category	Purpose	Example
Base types	Common fields (ID, timestamps)	BaseDocument
Enums	Fixed set of values	UserRole: "admin" | "user"
Interfaces	Object shapes	IUser, IBook
Document types	Mongoose + methods	IUserDocument
Request types	Express extended	AuthenticatedRequest
Response types	API response shapes	ApiSuccessResponse

9.2 Create src/types/index.ts

Task: Define all type definitions

We'll start with all types in one file and split them later as needed.

1. Base Types

Create BaseDocument interface:

- Include MongoDB fields that all documents have
- Fields: _id (Types.ObjectId), createdAt (Date), updatedAt (Date)
- These are auto-added by Mongoose when `timestamps: true`

2. Enums (Status Types)

Create UserRole type:

- Type alias (not interface)
- Union type: "admin" | "user"
- Used for authorization

Why type alias instead of enum?

- Simpler in TypeScript
- Works better with Mongoose
- Easier to work with in JavaScript

3. User Types

Create IUser interface:

Extend BaseDocument and add:

Field	Type	Required	Default	Notes
firstName	string	✅	—	
lastName	string	✅	—	
email	string	✅	—	Unique
password	string	✅	—	Will be hashed
phone	string?	No	—	Optional
avatar	string?	No	—	URL, optional
role	UserRole	✅	"user"	admin or user

Create IUserDocument interface:

Purpose: Extends IUser with Mongoose document methods

Extend both:

- IUser (your interface)
- Document (from mongoose)

Add method:

- `comparePassword(candidatePassword: string): Promise<boolean>`

Why separate IUser from IUserDocument?

- IUser = data shape
- IUserDocument = data + methods
- Clean separation of concerns

4. Express Request Extensions

Create AuthenticatedRequest:

- Extend Request from express
- Add: user property (type: IUserDocument)
- Used after CheckAuth middleware runs

Create RequestWithParsedQuery:

- Extend Request from express
- Add: parsedQuery property (type: T, generic)
- Used after validation middleware

Why extend Request?

- Middleware adds properties to req
- TypeScript needs to know they exist
- Prevents "Property does not exist" errors

Part 5: API Response Types

Create ApiSuccessResponse interface:

Generic interface for success responses:

- success: true (literal type)
- message: string
- data: T (generic type parameter)

Create ApiErrorResponse interface:

For error responses:

- success: false (literal type)
- message: string
- error?: string (optional, dev mode only)

Why generic T?

- Different endpoints return different data
- Type-safe: `ApiSuccessResponse<User>` vs `ApiSuccessResponse<Book>`

Create PaginationMeta interface:

Pagination information:

- total: number (total records)
- page: number (current page)
- limit: number (records per page)
- totalPages: number (calculated)

Create ApiPaginatedResponse interface:

For paginated list responses:

- success: true
- data: T[] (array of items)
- pagination: PaginationMeta
- message?: string (optional)

10. Utility Layer

10.1 Understanding the Utility Layer

Purpose: Reusable helper functions with no dependencies

Characteristics:

- Pure functions (no side effects)
- No model imports
- No service imports
- Can be used anywhere

What we'll build:

1. Logger (structured logging)
2. Helper functions (utilities)

10.2 Logger — src/utils/logger.ts

Task: Create structured logging system

Why logging?

- Debug issues in production
- Track application behavior
- Monitor errors and warnings
- Structured data for analysis

Step 1: Define types

Create LogLevel type:

- Union of: “ERROR” | “WARN” | “INFO” | “DEBUG”

Create LogEntry interface:

- timestamp: string (ISO format)
- level: LogLevel
- message: string
- meta?: Record<string, unknown> (optional metadata)

Step 2: Create internal log function

Function signature:

- `log(level: LogLevel, message: string, meta?: Record<string, unknown>): void`

Implementation requirements:

1. Build log entry object:
    - Current timestamp (new Date().toISOString())
    - Provided level, message
    - Include meta if provided
2. Format output based on environment:
    - Production: Single-line JSON (JSON.stringify(entry))
    - Development: Pretty-printed (JSON.stringify(entry, null, 2))
3. Route to correct stream:
    - ERROR → console.error()
    - All others → console.log()

Why different formatting?

- Development: Easy to read
- Production: Easy to parse by log aggregators

Step 3: Export public logger object

Create object with four methods:

- `error(message, meta?)` → calls log with "ERROR"
- `warn(message, meta?)` → calls log with "WARN"
- `info(message, meta?)` → calls log with "INFO"
- `debug(message, meta?)` → calls log with "DEBUG"

Usage pattern:

```
logger.info("Server started");
logger.error("Connection failed", { error: err.message });
logger.debug("Request received", { path: req.path });
```

Benefits of this logger:

- Consistent format everywhere
- Easy to upgrade to winston/pino later
- Structured metadata for filtering
- Timestamp on every log
- Environment-specific formatting

---

11. Database Connection

11.1 Understanding Mongoose Connection

What is Mongoose?

- ODM (Object Data Modeling) for MongoDB
- Provides schema validation
- Adds structure to MongoDB documents
- Includes middleware hooks
- TypeScript-friendly

Connection flow:

```
1. Load environment variables
2. Build connection string
3. Call mongoose.connect()
4. Handle success/failure
5. Return connection for server to use
```

11.2 Create src/services/db.ts

Task: Create database connection service

File purpose: Manage MongoDB connection lifecycle

1. Create connectDB function

Function signature:

- `async function connectDB()` (returns Promise)

Implementation requirements:

Part A: Enable debug mode in development

- Check if NODE_ENV === "development"
- If yes: `mongoose.set("debug", true)`
- This logs all queries to console

Why debug mode?

- See actual MongoDB queries
- Debug slow queries
- Verify query structure
- Only in development (too verbose for production)

Part C: Connect to MongoDB

Use try-catch block:

Try block:

1. Call `mongoose.connect()`
   - First parameter: process.env.MONGODB_URI
   - Second parameter: options object (`dbName` + optional `auth` object with `username` / `password`)
   - Use await
2. Log success with logger.info()
   - Include: host and database name from connection
3. Return the connection

Catch block:

1. Log error with logger.error()
   - Include: error message, URI (not password!)
2. Re-throw the error (let caller handle)

Why re-throw?

- Server needs to know connection failed
- Can't start server without database
- Will trigger process.exit in index.ts

Why separate the DB service?

- Enables testing with different DB connections
- Allows connection retry logic
- Environment-specific configuration

---

12. Authentication & User Management

12.1 User Model

1 Understanding Mongoose Models

Before writing the model, understand the Mongoose architecture:

```
Schema (Blueprint)          Model (Factory)           Document (Instance)
┌──────────────┐            ┌──────────────┐          ┌──────────────┐
│ Define:      │            │ Based on     │          │ Actual data  │
│ - Fields     │  ────────> │ schema       │ ───────> │ in MongoDB   │
│ - Types      │            │              │          │              │
│ - Validation │            │ Creates/     │          │ Can call     │
│ - Methods    │            │ queries docs │          │ methods      │
│ - Hooks      │            │              │          │              │
└──────────────┘            └──────────────┘          └──────────────┘
```

Key concepts:

Concept	Purpose	Example
Schema	Defines structure	Field types, validators, defaults
Model	Creates documents	`userModel.create()`, `userModel.find()`
Document	Single instance	A specific user with methods
Hook	Middleware on events	Hash password before saving
Method	Document functions	`user.comparePassword()`

2 Create src/models/user.ts

Task: Create User model with password hashing and validation

File purpose: Define how users are stored in MongoDB Create a new Schema with type parameter `<IUserDocument>`.

Define these fields:

Field	Type	Required	Default	Notes
`firstName`	String	✅	—	Trimmed
`lastName`	String	✅	—	Trimmed
`email`	String	✅	—	`unique: true`, lowercase
`password`	String	✅	—	Hashed via pre-save hook
`phone`	String	No	—	
`avatar`	String	No	—	URL
`role`	String	No	`"user"`	Enum: `["admin", "user"]`

Schema options: Add schema options (second parameter to Schema constructor):

- `timestamps: true` — Auto-manages `createdAt` / `updatedAt`
- `toJSON: { versionKey: false }` — Removes `__v` from responses

Create indexes

Index 1: Email unique index

Create on email field:

- Field: `{ email: 1 }`
- Options: `{ unique: true }`

Why explicit?

- Already created by unique: true in schema
- We define it explicitly for clarity
- Good documentation

Index 2: index for role queries

Create on role :

- Fields: `{ role: 1 }`
- No unique constraint

Use case:

- Query: "Find all admins"
- Query: "Find all users"
- Faster than filtering after fetch

Index direction (1 vs -1):

- 1 = ascending
- -1 = descending

Pre-save hook for password hashing

Task: Hash password automatically before saving

Hook setup:

- Event: "save"
- Timing: "pre" (before save)
- Function: async (use `function` not arrow for `this` context)

Pre-save hook:

```
// If password is new or changed:
//   this.password = await bcrypt.hash(this.password, 10)
```

Understanding bcrypt:

What is salting?

```
Without salt:
password → hash
"hello123" → "abc123..." (same every time)

With salt:
password + random salt → hash
"hello123" + "x7kp2" → "def456..." (unique)
"hello123" + "m9qz1" → "ghi789..." (different!)

Same password → different hash each time!
```

Salt rounds (cost factor):

Rounds	Time	Security Level	Recommendation
8	~40ms	Minimum	Testing only
10	~100ms	Good	✅ Production default
12	~300ms	High	High-security apps
14	~1s	Very high	Overkill for most

Why 10 rounds?

- Good security vs speed balance
- Industry standard
- Fast enough for user experience
- Secure against brute force

Each round doubles the computational cost.

Instance method for password comparison

Task: Add method to verify passwords

Method definition:

Name: `comparePassword` Parameters: `candidatePassword: string` Returns: `Promise<boolean>`

Implementation:

Wrap in try-catch:

Try block:

1. Use: `bcrypt.compare(candidatePassword, this.password)`
2. Return the result (boolean)
3. bcrypt handles salt extraction automatically

Catch block:

- Return false (comparison failed)

Why this method?

- Encapsulates password verification logic
- Type-safe (TypeScript knows method exists)
- Consistent across application
- Hides bcrypt implementation details

How it works:

```
1. User logs in with plain password: "password123"
2. We have stored hash: "$2b$10$xyz..."
3. bcrypt.compare():
   - Extracts salt from stored hash
   - Hashes provided password with same salt
   - Compares hashes
   - Returns true if match, false if not
```

Important: NEVER compare plain text passwords!

```
// ❌ WRONG - Insecure
if (password === user.password) { }

// ✅ CORRECT - Secure
if (await user.comparePassword(password)) { }
```

Create and export model

Create model:

- First parameter: “users” (collection name)
- Second parameter: userSchema
- Type parameter: `<IUserDocument>`

Export as: `userModel`

Naming conventions:

- Collection: lowercase plural (“users”)
- Model variable: camelCase + Model suffix (“userModel”)
- File name: singular (“user.ts”)

12.2. User Validation

1. Create src/validation/utils.ts

Task: Create shared validation schemas

File purpose: Reusable validation patterns

Schema 1: mongoIdSchema

Purpose: Validate MongoDB ObjectId format

Requirements:

- Type: string
- Pattern: 24 hexadecimal characters
- Regex: `/^[a-f\d]{24}$/i`
- Error message: “Invalid MongoDB ObjectId format”

Use Zod methods:

- `z.string()`
- `.regex(pattern, message)`

Usage: URL parameters (`:id`), foreign keys

Schema 2: paginationSchema

Purpose: Validate and transform pagination query params

Object schema with these fields: Field: page

- Input type: string (query params are always strings)
- Optional: true
- Default: "1"
- Transform: Convert to Number
- Validation: Must be an integer >= 1
- Error: "Page must be a positive integer"

Field: limit

- Input type: string
- Optional: true
- Default: "10"
- Transform: Convert to Number
- Validation: Must be an integer >= 1 and <= 100
- Errors: "Limit must be a positive integer" / "Limit cannot exceed 100"

Zod methods to use:

- `z.object({ ... })`
- `z.string()`
- `.optional()`
- `.default(value)`
- `.transform(fn)`
- `.pipe(schema)`
- `z.number()`
- `.int()`
- `.min(n, message)`
- `.max(n, message)`

Why transform?

```
Input:  req.query = { page: "2", limit: "10" }
Output: parsedQuery = { page: 2, limit: 10 }

Now you can do math:
skip = (page - 1) * limit  // Works!
```

Why max limit of 100?

- Prevents abuse: `?limit=999999`
- Controls server load
- Reasonable page size
- Can be adjusted per endpoint

Schema 3: idParamsSchema

Purpose: Validate URL parameters with ID

Object schema:

- Field: `id`
- Use: mongoIdSchema (defined above)

Usage:

```
Route: GET /users/:id
Validate: req.params
Result: { id: "507f1f77bcf86cd799439011" }
```

2.Create src/validation/user.ts

Task: Create all user-related validation schemas

File purpose: Request validation for auth and user endpoints

Step 1: Import dependencies

You'll need:

- `z` from zod

Step 2: Password validation (progressive approach)

Create basicPasswordSchema:

- Type: string
- Minimum length: 6
- Error message: "Password must be at least 6 characters"

Why start simple?

- Easier testing during development
- Can use "123456" for quick tests
- Upgrade to strong passwords later

Comment out strongPasswordSchema (for later):

Create commented-out schema:

- Regex pattern: `/^(?=.*?[A-Z])(?=.*?[a-z])(?=.*?[0-9]).{8,}$/`
- Error message: "Password must be 8+ characters with uppercase, lowercase, and number"

Pattern explanation:

```
^                Start of string
(?=.*?[A-Z])     At least one uppercase
(?=.*?[a-z])     At least one lowercase
(?=.*?[0-9])     At least one digit
.{8,}            Minimum 8 characters
$                End of string
```

When to upgrade:

- After initial testing
- Before production deployment
- Update seed data passwords
- Uncomment strong schema
- Use it instead of basic

Step 3: Assign current password schema

Create variable:

- Name: `passwordSchema`
- Value: `basicPasswordSchema`

Later change to: `strongPasswordSchema`

Step 4: Login schema

Object schema with these fields:

Field: email

- Type: string
- Validation: `.email()` with message "Invalid email format"
- Transform: `.toLowerCase()`
- Transform: `.trim()`

Field: password

- Use: passwordSchema (defined above)

Why toLowerCase?

- "John@Example.com" → "john@example.com"
- Consistent with model
- Prevents duplicate accounts with different casing

Why trim?

- " john@example.com " → "john@example.com"
- Users often paste with spaces
- Clean input automatically

Step 5: User registration schema

Extend login schema with additional fields:

Field: firstName

- Type: string
- Min length: 2 with message
- Max length: 50 with message
- Transform: trim

Field: lastName

- Type: string
- Min length: 2 with message
- Max length: 50 with message
- Transform: trim

Field: phone

- Type: string
- Optional: true
- Transform: Convert empty string to null
  - Pattern: `.transform((val) => val || null)`

Field: avatar

- Type: url
- Optional: true
- Transform: Convert empty string to null

Zod pattern:

```
// Base schema
const loginSchema = z.object({
  email: ...,
  password: ...
});

// Extended schema
const userSchema = loginSchema.extend({
  firstName: ...,
  lastName: ...,
  phone: ...,
  avatar: ...
});
```

Step 6: Full user schema (admin)

Extend userSchema with role field:

Field: role

- Type: enum
- Values: ["admin", "user"]
- Optional: true (defaults to "user" in model)

Step 7: Profile update schema

Object schema (all fields optional):

Fields:

- firstName: optional, min 2, max 50, trim
- lastName: optional, min 2, max 50, trim
- phone: optional, transform to null
- avatar: optional, URL validation, transform to null

Why all optional?

- Partial updates
- User can update just name
- Or just phone
- Or just avatar
- No need to send all fields

Note: Password NOT included here (separate endpoint)

Step 8: Change password schema

Object schema:

Field: currentPassword

- Type: string
- Min length: 1
- Error: "Current password is required"

Field: newPassword

- Use: passwordSchema

Why require current password?

- Security: Verify user knows current password
- Prevents: Someone using logged-in session to change password
- Common attack: Session hijacking → change password
- Protection: Must know current password to change

Step 9: Export type inference

Create TypeScript types from schemas:

Export these types using Zod's inference:

- `LoginInput` from loginSchema
- `UserInput` from userSchema
- `ProfileUpdateInput` from profileUpdateSchema
- `ChangePasswordInput` from changePasswordSchema

Zod inference pattern:

```
export type LoginInput = z.infer<typeof loginSchema>;
```

Why infer?

- DRY (Don't Repeat Yourself)
- Schema is single source of truth
- Types automatically match validation
- Change schema → types update automatically

Usage in handlers:

```
export const login = async (req: { body: LoginInput }, res: Response) => {
  const { email, password } = req.body;
  // TypeScript knows email and password exist and are strings
};
```

3. Validation Strategy

Two-layer validation:

```
Layer 1: Mongoose Schema (Database level)
- Basic type checking
- Required fields
- Unique constraints
- Default values

Layer 2: Zod Schemas (Application level)
- Format validation (email, URL)
- Complex rules (password strength)
- Transformations (trim, lowercase)
- Custom validation (regex)
```

Why both?

- **Mongoose:** Last line of defense, catches programming errors
- **Zod:** Better error messages, transforms, catches user errors early

Example flow:

```
1. User submits: { email: " JOHN@EXAMPLE.COM ", password: "pass" }
2. Zod validates:
   - Transforms email: "john@example.com"
   - Validates password length: FAIL
   - Returns error: "Password must be at least 6 characters"
3. (If Zod passed) Mongoose validates:
   - Checks required fields
   - Checks unique email
   - Validates field types
4. (If both passed) Document saved
```

12.3. Error Handling System

1. Create src/middlewares/errorHandler.ts

Task: Create comprehensive error handling middleware

File purpose: Catch and format all errors

Step 1: Import dependencies

You'll need:

- Express types: `Request`, `Response`, `NextFunction`
- `StatusCodes` from `http-status-codes`
- `logger` from your utils

Step 2: Define MongooseError interface

Create a local interface that extends `Error` with these optional properties:

- `code`: number
- `keyPattern`: Record with string keys and unknown values
- `errors`: Record with string keys, each containing an object with a `message` string
- `path`: string
- `value`: unknown

This avoids importing the full mongoose package just for types.

Step 3: Create errorHandler function

Function signature:

- Parameters: `err: Error | MongooseError`, `req: Request`, `res: Response`, `_next: NextFunction`
- Return: void
- Note: 4 parameters are required for Express to recognize it as error middleware
- Note: `_next` prefix indicates intentionally unused parameter

Step 4: Log every error at the top

Before any error type checks, log the error using `logger.error()` with:

- Message: `"Error occurred"`
- Metadata object containing: `error` (err.message), `stack` (err.stack), `path` (req.path), `method` (req.method)

This runs for all errors unconditionally.

Step 5: Handle each error type with if/return blocks

Use separate `if` blocks, each ending with `return` to stop execution.

Error Type 1: Mongoose ValidationError

Check: `err.name === "ValidationError"`

If true:

- Cast `err` to `MongooseError`
- Respond with status: 400 (BAD_REQUEST)
- Body:
  - `success`: false
  - `message`: "Validation error"
  - `error`: if `mongoErr.errors` exists, map `Object.values()` to extract each `.message`; otherwise fall back to `err.message`
- Return

Error Type 2: MongoDB Duplicate Key

Check: Cast `err` to `MongooseError` outside the block (reuse for later checks too), then check `mongoErr.code === 11000`

If true:

- Extract field name: if `mongoErr.keyPattern` exists, get first key from `Object.keys()`; otherwise fall back to `"field"`
- Respond with status: 409 (CONFLICT)
- Body:
  - `success`: false
  - `message`: template string — `"Duplicate value for ${field}"`
- Return

Error Type 3: Mongoose CastError

Check: `err.name === "CastError"`

If true:

- Respond with status: 400 (BAD_REQUEST)
- Body:
  - `success`: false
  - `message`: template string using `mongoErr.path` and `mongoErr.value` — `"Invalid ${path}: ${value}"`
- Return

Error Type 4: JsonWebTokenError

Check: `err.name === "JsonWebTokenError"`

If true:

- Respond with status: 401 (UNAUTHORIZED)
- Body:
  - `success`: false
  - `message`: "Invalid token"
- Return

Error Type 5: TokenExpiredError

Check: `err.name === "TokenExpiredError"`

If true:

- Respond with status: 401 (UNAUTHORIZED)
- Body:
  - `success`: false
  - `message`: "Token expired"
- Return

Step 6: Default catch-all (no check needed)

Reached only if no earlier block matched.

Respond with status: 500 (INTERNAL_SERVER_ERROR)

Body:

- `success`: false
- `message`: if `NODE_ENV === "development"`, use `err.message`; otherwise use `"Internal server error"`
- Conditionally spread `stack` property: only include it when `NODE_ENV === "development"`, using spread with short-circuit `&&`

---

2. Understanding Error Handler Placement

Critical: Error handler must be LAST middleware!

```
// ❌ WRONG - Routes after error handler
app.use(errorHandler);
app.use("/users", userRouter);

// ✅ CORRECT - Error handler is last
app.use("/users", userRouter);
app.use(errorHandler);
```

Why?

- Express uses 4-parameter signature to detect error handlers
- Error handlers catch errors from previous middleware
- No middleware runs after error handler
- Must be registered last

12.4. Authentication Middleware

1 Understanding JWT Authentication

What is JWT?

JSON Web Token - A compact way to securely transmit information.

Structure:

```
header.payload.signature

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJfaWQiOiI1MDdmMWY3N2JjZjg2Y2Q3OTk0MzkwMTEiLCJpYXQiOjE3MTA1MDAwMDB9.
signature-here
```

Parts:

1. **Header:** Algorithm and type
2. **Payload:** Data (user ID, issued time, expiration)
3. **Signature:** Verifies token wasn't tampered with

JWT Authentication Flow:

```
┌──────────────┐
│   Client     │
└──────────────┘
       │
       │ 1. POST /auth/login
       │    { email, password }
       ▼
┌──────────────┐
│   Server     │ 2. Verify credentials
└──────────────┘ 3. Generate JWT
       │         jwt.sign({ _id }, SECRET, { expiresIn: "7d" })
       │ 4. Return { user, token }
       ▼
┌──────────────┐
│   Client     │ 5. Store token (localStorage)
└──────────────┘
       │
       │ 6. Subsequent requests:
       │    GET /users/profile
       │    Authorization: Bearer <token>
       ▼
┌──────────────┐
│   Server     │ 7. CheckAuth middleware
└──────────────┘    - Extract token
       │             - Verify signature
       │             - Decode payload
       │             - Lookup user
       │             - Attach to req.user
       │ 8. Handler accesses req.user
       ▼
┌──────────────┐
│  Response    │
└──────────────┘
```

2. Create src/middlewares/auth.ts

Task: Create JWT authentication middleware

File purpose: Verify tokens and load user data

Step 1: Define JWT payload interface

Create interface locally in this file:

JWTPayload interface:

- _id: string (required)
- iat?: number (optional, issued at timestamp)
- exp?: number (optional, expiration timestamp)

Note: iat and exp are auto-added by jwt.sign()

Step 2: Create CheckAuth middleware

The JWT verification middleware:

1. Get `authorization` header
2. Check if exists and starts with "Bearer "
3. If not: throw UnauthorizedError("No token provided")
4. Split `"Bearer <token>"` to get the token
5. If empty: throw UnauthorizedError("No token provided") **Header format:**

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
               ^^^^^^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
               prefix token
```

6. Verify the JWT using `jwt.verify()` with `AUTH_SECRET` **What verify does:**

- Checks signature matches
- Checks token hasn't expired
- Returns payload if valid
- Throws error if invalid

7. Decode the `_id` from the token payload
8. Find the user by ID: `userModel.findById(decoded._id).select("-password")`
9. Check that user exists
10. Attach user to `(req as AuthenticatedRequest).user`
11. Call `next()` on success, return `401` on any failure

Result:

- Subsequent middleware and handlers can access `req.user`
- Type-safe: TypeScript knows user exists
- No password included

Step 3: Create isAdmin middleware

- Must be used **after** `CheckAuth`
- Checks `req.user.role === "admin"`
- Returns `403 Forbidden` if not an admin

Middleware Usage Patterns

Pattern 1: Protected route (any authenticated user)

```
router.get("/profile", CheckAuth, getProfile);
```

Flow:

1. CheckAuth verifies token
2. CheckAuth loads user
3. CheckAuth calls next()
4. getProfile accesses req.user

Pattern 2: Admin-only route

```
router.get("/users", CheckAuth, isAdmin, getUsers);
```

Flow:

1. CheckAuth verifies token
2. CheckAuth loads user
3. CheckAuth calls next()
4. isAdmin checks user.role
5. If admin: calls next()
6. If not admin: throws ForbiddenError
7. getUsers accesses req.user

Critical: CheckAuth MUST come before isAdmin!

Pattern 3: Public route

```
router.get("/books", getBooks);
```

No authentication required. Anyone can access.

12.5 Validation Middleware — `src/middlewares/validations.ts`

Three middleware factories:

Factory	Validates	On Success	On Failure
`validateBodySchema(schema)`	`req.body`	`next()`	400 + Zod errors
`validateQuerySchema(schema)`	`req.query`	Attaches `req.parsedQuery`, calls `next()`	400 + Zod errors
`validateParamsSchema(schema)`	`req.params`	`next()`	400 + Zod errors

> ⚠️ **Important:** The query middleware stores transformed data in `req.parsedQuery` because Zod's `transform` changes types (e.g., string `"1"` → number `1`). Handlers should read from `parsedQuery`, not `req.query`.

12.6. Auth & User Handlers

1. Understanding Handlers

Handlers (also called controllers) contain business logic:

```
Router → Middleware → Handler → Model → Database
  ↓          ↓           ↓
Define    Validate   Process
route     input      request
```

Handler responsibilities:

1. Extract validated data from request
2. Perform business logic (create user, verify password, etc.)
3. Interact with database via models
4. Format and send response
5. Throw errors when something goes wrong

---

2. Create src/handlers/auth.ts

File purpose: Handle authentication operations (register, login, check auth status)

Handler 1: register Route: POST /auth/register
Purpose: Create a new user account
Input: UserInput from req.body (validated by middleware)

Steps:

1. Create user using `userModel.create(req.body)`
   - Note: Password is automatically hashed by the pre-save hook
2. Sign a JWT for immediate login after registration
3. Build response object:
   - success: true
   - message: "User registered successfully"
   - data: { user: sanitizeUser(user), token }
4. Send response with status 201 (CREATED)

Error handling:

- Duplicate email → Mongoose throws error with code 11000
- Let error handler middleware catch it (already configured)

Handler 2: login

Route: POST /auth/login
Purpose: Authenticate user and return JWT
Input: LoginInput from req.body (email, password)

Steps:

1. Extract email and password from body
2. Find user by email
3. Check if user exists:
   - If not → throw UnauthorizedError("Invalid email or password")
4. Verify password:
   - Call `user.comparePassword(password)`
   - If false → throw UnauthorizedError("Invalid email or password")
   - Important: Use same error message for both cases (security best practice)
5. Create JWT payload with `{ _id, createdAt }`, sign it with `AUTH_SECRET`, set expiry from `JWT_EXPIRES_IN`
6. Return safe user data (password excluded) and the token
7. Send response with status 200 (OK)

Security considerations:

- Never reveal whether email exists or password is wrong
- Always use generic "Invalid email or password"
- Check account status to prevent deactivated users from logging in

---

Handler 3: checkUser Route: GET /auth/check
Purpose: Verify token validity and return current user
Requires: CheckAuth middleware (must be called before this handler)

Steps:

1. Access user from `req.user` (already populated by CheckAuth middleware)
2. Build response:
   - success: true
   - message: "Authenticated"
   - data: req.user
3. Send response with status 200 (OK)

Note: This is a simple handler because CheckAuth does all the heavy lifting

- Simple "am I logged in?" endpoint

---

3. User Handlers: Create src/handlers/user.ts

File purpose: Handle user-related operations (profile, CRUD operations)

Admin CRUD: `getUsers`, `createUser`, `getUserById`, `updateUser`, `deleteUser`

- Standard Mongoose CRUD with error handling
- Duplicate email detection on create
- Password excluded from responses

User self-service:

- `getProfile` — Return current user with computed stats
- `updateProfile` — Allow updating firstName, lastName, phone, avatar; sanitize empty strings to null
- `changePassword` — Verify current password, then update (triggers pre-save hash hook)

12.7. Auth & User Routes

1 Critical: Understanding Route Order

⚠️ MOST IMPORTANT CONCEPT FOR ROUTES

Express matches routes **top to bottom**, first match wins.

The Problem:

Parameterized routes like `/:id` match **ANY string**, including literal path names.

Example of wrong order:

```
router.get("/:id", getUserById);      // Defined first
router.get("/profile", getProfile);   // Defined second

User visits: GET /users/profile

Express checks routes in order:
1. Check /:id → "profile" matches! (it's a string)
2. Calls getUserById with req.params.id = "profile"
3. Tries to find user with ID "profile"
4. MongoDB CastError: "profile" is not a valid ObjectId
5. Error response

The /profile route is never reached!
```

Solution: Specific routes BEFORE parameterized routes:

```
router.get("/profile", getProfile);   // Specific path first
router.get("/:id", getUserById);      // Generic path last

User visits: GET /users/profile

Express checks routes in order:
1. Check /profile → Exact match! ✓
2. Calls getProfile
3. Success!

User visits: GET /users/abc123

Express checks routes in order:
1. Check /profile → Doesn't match
2. Check /:id → Matches! ✓
3. Calls getUserById with req.params.id = "abc123"
4. Success!
```

Rule: Static/named routes BEFORE dynamic/:param routes

---

2. Create src/routers/auth.ts

File purpose: Define authentication routes

- Create router: `const router = Router()`

Define routes:

Prefix: `/auth`

Method	Path	Middleware	Handler	Access
POST	`/login`	`validateBodySchema(loginSchema)`	`login`	Public
POST	`/register`	`validateBodySchema(userSchema)`	`register`	Public
GET	`/check`	`CheckAuth`	`checkUser`	Private

3. Create src/routers/user.ts

File purpose: Define user management routes

- Create router: `const router = Router()`

⚠️ CRITICAL: Route definition order matters!

Prefix: `/users`

Protected (authenticated user): User's Own Data (Protected) - MUST BE FIRST

Method	Path	Middleware	Handler
GET	`/profile`	`CheckAuth`	`getProfile`
PUT	`/profile`	`CheckAuth`, `validateBodySchema(profileUpdateSchema)`	`updateProfile`
PUT	`/change-password`	`CheckAuth`, `validateBodySchema(changePasswordSchema)`	`changePassword`

Admin routes:

Method	Path	Middleware	Handler
GET	`/`	`CheckAuth`, `isAdmin`	`getUsers`
POST	`/`	`CheckAuth`, `isAdmin`	`createUser`
GET	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`	`getUserById`
PUT	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`	`updateUser`
DELETE	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`	`deleteUser`

---

4. Understanding Middleware Chains

Each route can have multiple middleware functions that execute in sequence:

```
router.post(
  "/register",
  middleware1,  // Runs first
  middleware2,  // Runs second
  handler       // Runs last (if no errors)
);
```

Example: POST /users (admin create user)

```
router.post(
  "/",
  CheckAuth,                          // 1. Verify JWT token
  isAdmin,                            // 2. Check admin role
  validateBodySchema(fullUserSchema),  // 3. Validate body
  createUser                          // 4. Create user
);
```

Execution flow:

```
Request arrives
   ↓
CheckAuth:
  - Extract token from Authorization header
  - Verify token with jwt.verify()
  - Load user from database
  - Attach to req.user
  - Call next()
   ↓
isAdmin:
  - Check req.user.role === "admin"
  - If yes: call next()
  - If no: throw ForbiddenError
   ↓
validateBodySchema:
  - Parse req.body with Zod schema
  - If valid: call next()
  - If invalid: throw ValidationError
   ↓
createUser:
  - Create user in database
  - Generate token
  - Send response
```

If any middleware throws error:

```
Middleware throws error
   ↓
(skip remaining middleware)
   ↓
Error handler catches it
   ↓
Send error response
```

Common mistake:

```
// ❌ Wrong order
router.get("/", handler, CheckAuth);
// Handler runs without authentication!

// ✅ Correct order
router.get("/", CheckAuth, handler);
// Authentication happens first
```

---

13. Express App Assembly & Server Entry

13.1 Create src/app.ts

File purpose: Configure Express application (middleware, routes, error handling) **Important:** This file does NOT start the server. It only configures it.

Step 1: Create Express instance

```
const app = express();
```

Step 2: Register global middleware (ORDER MATTERS)

Middleware 1: helmet()

- Purpose: Sets security-related HTTP headers
- Protects against: XSS, clickjacking, MIME sniffing
- Must be first to apply to all requests
- Example headers: X-Content-Type-Options, X-Frame-Options

Middleware 2: cors()

- Purpose: Enable Cross-Origin Resource Sharing
- Configuration:
  - origin: process.env.FRONTEND_DOMAIN
  - credentials: true (allows cookies/auth headers)
- Why needed? Browser blocks requests from different origins by default
- Example: Frontend on localhost:5173, API on localhost:3000

Middleware 3: express.json()

- Purpose: Parse JSON request bodies
- Limit: 10mb (prevents large payload attacks)
- Converts JSON string to JavaScript object
- Populates req.body

Middleware 4: express.urlencoded()

- Purpose: Parse URL-encoded form data
- Extended: true (allows nested objects)
- Limit: 10mb
- Handles: application/x-www-form-urlencoded

Why this order?

1. Security first (helmet, cors)
2. Body parsing second (needed by routes)
3. Routes third (business logic)
4. Error handling last (catches all errors)

Step 3: Mount route handlers

```
app.use("/auth", authRouter);
```

- All routes in authRouter are prefixed with /auth
- Example: /auth/login, /auth/register

```
app.use("/users", userRouter);
```

- All routes in userRouter are prefixed with /users
- Example: /users/profile, /users/:id

Step 4: Health check endpoint

Create: GET /health

- Purpose: Verify server is running
- Used by: Monitoring tools, load balancers, deployment systems
- Response:
  - success: true
  - message: "Server is running"
  - data: { timestamp: current ISO string }
- Status: 200 OK

Why needed?

- Quick way to check if server is alive
- No authentication required
- No database access (fast response)
- Useful for debugging deployment issues

Step 5: 404 handler for undefined routes

Create catch-all route: app.use("*", handler)

- Must be after all valid routes
- Catches any unmatched paths
- Response:
  - success: false
  - message: "Route not found"
- Status: 404 NOT_FOUND

Why wildcard "*"?

- Matches any path not handled by previous routes
- Example: GET /nonexistent → 404 response

Step 6: Global error handler

Register: app.use(errorHandler)

- MUST be last middleware
- Takes 4 parameters: (err, req, res, next)
- Catches all errors from routes and middleware
- Formats errors into consistent API responses

Why last?

- Express calls error handlers when next(error) is called
- Must be after all routes to catch their errors
- No middleware after this point (error handlers are terminal)

Step 7: Export app

```
export default app;
```

- Don't call app.listen() here!
- Separation allows:
  - Testing without starting server
  - Different startup logic per environment
  - Multiple server instances (HTTP, HTTPS)

13.2 Understanding Middleware Execution Order

Request flow diagram:

```
HTTP Request
   ↓
helmet() → Sets security headers
   ↓
cors() → Checks origin, adds CORS headers
   ↓
express.json() → Parses JSON body
   ↓
express.urlencoded() → Parses form data
   ↓
Route matching:
   ├→ /auth/* → authRouter
   ├→ /users/* → userRouter
   ├→ /health → Health check
   └→ /* → 404 handler
   ↓
errorHandler() → Catches any errors
   ↓
HTTP Response
```

Example request:

```
POST /auth/login
Content-Type: application/json
{ "email": "...", "password": "..." }

1. helmet() → Adds security headers
2. cors() → Checks if frontend domain is allowed
3. express.json() → Parses JSON body to req.body
4. Matches /auth route → Goes to authRouter
5. authRouter matches /login → Goes to login handler
6. If error → Goes to errorHandler
7. If success → Sends response
```

---

14. Server Entry Point

14.1 Create src/index.ts

File purpose: Start the HTTP server and manage its lifecycle

⚠️ CRITICAL FIRST LINE:

`import "dotenv/config";`

This MUST be the first import before anything else!

Why?

- Loads environment variables from .env file
- Makes `process.env.*` available to all modules
- Other files depend on these variables (AUTH_SECRET, MONGODB_URI, etc.)
- If imported late, variables won't be available

Import dependencies:

- `"dotenv/config"` (must be first!)
- `app` from `./app.js`
- `connectDB` from `./services/db.js`
- `logger` from your utils

Define constants:

- `PORT`: read from `process.env.PORT`, fallback to `3000` if not set

Start the server using promise chaining:

Call `connectDB()` directly (no wrapper function), then chain `.then()` and `.catch()`.

In the `.then()` callback:

- Log `"Connected to MongoDB"` using `logger.info()`
- Call `app.listen(PORT, callback)`
- In the listen callback, log two separate info messages:
  - Server running message including the PORT value
  - Environment message using `process.env.NODE_ENV` with fallback to `"development"`

In the `.catch()` callback:

- Receives an `error` parameter
- Log failure using `logger.error()` with message `"Failed to start server:"` and metadata object containing the error
- Call `process.exit(1)` to terminate with failure code

---

15. Part 2 Dependencies & Structure Evolution

15.1 Install New Packages

Task: Add packages needed for this sprint

Development dependencies to install:

```
npm install -D morgan @types/morgan
```

Package	Purpose
`morgan`	HTTP request logger (logs incoming requests)
`@types/morgan`	TypeScript types for morgan

Why morgan?

- See all incoming requests in console
- Helpful for debugging
- Shows: method, path, status, response time
- Example: `GET /books 200 45ms`

Note: `http-status-codes` should already be installed from Part 1.

15.2 Update Express App with Morgan

Task: Add request logging to app.ts

In src/app.ts:

Step 1: Import morgan

```
import morgan from "morgan";
```

Step 2: Add middleware (AFTER helmet and cors, BEFORE routes)

Configuration:

- Environment check: Only in development
- Format: "dev" (colored output)

Pattern:

```
if (process.env.NODE_ENV === "development") {
  app.use(morgan("dev"));
}
```

Morgan formats:

Format	Output Example	Use Case
`dev`	`GET /books 200 45ms` (colored)	Development
`combined`	Apache combined log format	Production
`tiny`	Minimal output	Production
`common`	Common log format	Production

Why only in development?

- Production uses dedicated logging services
- Morgan output too verbose for production
- Structured logging (your logger) better for production

15.3. Update Type System

1 Split types/index.ts

Task: Organize types into separate files

Current situation:

- All types in `types/index.ts` (getting long)

New structure:

```
types/
├── common.ts           # Enums, base types
├── express.ts          # Request extensions
├── responses.ts        # API response types
└── models/
    ├── user.ts         # User types
    ├── book.ts         # NEW - Book types
    ├── author.ts       # NEW - Author types
    ├── category.ts     # NEW - Category types
    └── rating.ts       # NEW - Rating types
```

2. Create types/common.ts

Task: Move and add common types

Content to move from index.ts:

- `BaseDocument` interface
- `UserRole` type

New types to add:

BookStatus type (union):

- Values: "in-shelf" | "out-of-stock"
- Auto-calculated based on availableStock

Why these statuses?

- "in-shelf": availableStock > 0
- "out-of-stock": availableStock === 0
- Simple and clear for users

---

3. Create types/models/category.ts

Task: Define category types

Type 1: ICategory interface

Extends: BaseDocument

Fields:

- name: string (required)

Extremely simple:

- Categories are just names
- Fiction, Non-Fiction, Science, etc.

---

4. Create types/models/author.ts

Task: Define author types

Type 1: IAuthor interface

Extends: BaseDocument

Fields:

Field	Type	Required
`name`	`string`	Yes
`bio`	`string`	No (`?`)

Simple by design:

- Authors are relatively simple entities
- More fields can be added later

---

5. Create types/models/book.ts

Task: Define book-related types

Type 1: BookPrice interface

Purpose: Structured pricing with optional discount

Fields:

Field	Type	Required
`original`	`number`	No (`?`)
`current`	`number`	Yes

Usage:

```
// Regular price
{ current: 29.99 }

// Discounted price
{ original: 39.99, current: 29.99 }
```

Type 2: IBook interface

Extends: BaseDocument

Core fields:

Field	Type	Required	Notes
title	string	✅	Book name
author	ObjectId	✅	Reference to authors collection
serialNumber	string	✅	Unique, format: BK-XXXXX
price	BookPrice	✅	Sub-document
rentalPrice	number	No	Default 0
lateFeePerDay	number	No	Default 1
totalStock	number	No	Default 0
availableStock	number	No	Default 0
status	BookStatus	No	Auto-calculated
description	string	✅	Book description
cover	string	No	Cover image URL
keywords	string[]	No	For text search
category	ObjectId	No	Reference to categories

Why serialNumber?

- Unique identifier for physical books
- Format: BK-00001, BK-00002, etc.
- Easy to reference and scan

Stock fields explained:

- **totalStock:** Total copies owned
- **availableStock:** Copies available to borrow/buy
- **Relationship:** availableStock ≤ totalStock

---

6. Create types/models/rating.ts

Task: Define rating types

Type 1: IRating interface

Extends: BaseDocument

Fields:

Field	Type	Required	Notes
`ratedBy`	`Types.ObjectId`	Yes	Ref to users
`bookId`	`Types.ObjectId`	Yes	Ref to books
`rating`	`number`	Yes	1–5

Constraints:

- One rating per user per book (unique compound index)
- Rating must be 1, 2, 3, 4, or 5

---

7. Update types/express.ts

Task: Add new query parameter types

Add these interfaces:

BookIdParams:

```
{
  bookId: string;
}
```

SearchQuery:

```
{
  search?: string;
  category?: string;
}
```

SortQuery:

```
{
  sortBy?: string;
  sortOrder?: 1 | -1;
}
```

FilterQuery: Combine all three:

- PaginationQuery (from Part 1)
- SearchQuery
- SortQuery

---

8. Update User Types

Task: Add book-related fields to user

In types/models/user.ts:

Add UserBooks interface:

```
interface UserBooks {
  borrowed: Types.ObjectId[];
  read: Types.ObjectId[];
  favorites: Types.ObjectId[];
}
```

Update IUser interface:

Add fields:

- borrowLimit: number (default 3, minimum 1)
- fines: number (default 0, minimum 0)
- books: UserBooks

Why add to user model?

- Track user's book interactions
- Enable personalized recommendations
- Quick access to user's library

---

16. Models

16.1. Category Model

1. Create src/models/category.ts

Task: Create simple Category model

The simplest model:

- Just name and timestamps
- No references
- No special logic

Step 1: Create category schema

Field: name

Field	Type	Required	Constraints
`name`	String	Yes	`unique: true`, `trim: true`

Why unique?

- One "Fiction" category
- Prevents duplicates
- Creates automatic index

Step 3: Schema options

Standard options:

Option	Value
`timestamps`	`true`

Step 4: Create and export model

Model: "category" Export as: categoryModel

---

2. Category Design Decisions

Why simple?

- Categories are just labels
- No complex relationships
- Easy to maintain

Potential future enhancements:

- Parent categories (Fiction → Sci-Fi)
- Category images
- Category descriptions
- But start simple!

16.2. Author Model

1 Create src/models/author.ts

Task: Create Author model with virtual populate

Step 1: Create author schema

Simple schema with type `<IAuthor>`.

Field	Type	Required	Constraints
`name`	String	Yes	—
`bio`	String	No	—

Step 2: Schema options

Option	Value
`timestamps`	`true`
`toJSON`	`{ virtuals: true, versionKey: false }`
`toObject`	`{ virtuals: true, versionKey: false }`

> `virtuals: true` is required — without it, the `books` virtual field disappears from responses.

Step 3: Virtual populate for books

Task: Add virtual field that gets author's books

Virtual configuration:

- Field name: “books”
- Reference model: “books”
- Foreign field: “author” (field in Book model)
- Local field: “_id” (field in Author model)

> **How it works:** When you call `.populate("books")` on an author, Mongoose finds all documents in the `books` collection where `book.author === author._id` and attaches them to the `books` field.

```
const author = await AuthorModel.findById(id).populate("books");
// author.books = [book1, book2, book3]
```

Virtual vs Regular field:

Type	Stored in DB	Populated on query	Performance
Regular	✅ Yes	N/A	Fast read, slow write
Virtual	❌ No	✅ Yes	Slow read, fast write

Why virtual for books?

- Author doesn’t need to store book IDs
- Books already reference author
- Reverse lookup on demand
- Less data duplication

Step 5: Create and export model

model: “author” Export as: authorModel

---

2 Understanding Virtual Populate

How it works:

1. Author document:

```
{
  "_id": "author123",
  "name": "Isaac Asimov",
  "bio": "Science fiction author"
}
```

2. Book documents:

```
[
  { "_id": "book1", "title": "Foundation", "author": "author123" },
  { "_id": "book2", "title": "I, Robot", "author": "author123" }
]
```

3. Virtual populate query:

```
const author = await AuthorModel.findById("author123").populate("books");
```

4. Result:

```
{
  "_id": "author123",
  "name": "Isaac Asimov",
  "bio": "Science fiction author",
  "books": [
    { "_id": "book1", "title": "Foundation", "author": "author123" },
    { "_id": "book2", "title": "I, Robot", "author": "author123" }
  ]
}
```

MongoDB does:

- Find author by _id
- Find all books where author === author._id
- Attach to virtual field

16.3. Book Model

Book model is the most complex model because:

1. Multiple fields with validation
2. Sub-documents (price)
3. References (author, category, user)
4. Auto-calculated fields (status)
5. Text search indexes
6. Business logic (stock management)

1. Create src/models/book.ts

Task: Create Book Mongoose model

Step 1: Create book schema

Field Definitions:

Field 1: title

Configuration:

- type: String
- required
- trim: true

Index: Create text index (for search)

Field 2: author

Configuration:

- type: Schema.Types.ObjectId
- ref: "author"
- required

Why ref?

- Enables populate()
- Links to authors collection
- Maintains referential integrity

Field 3: serialNumber

Configuration:

- type: String
- required
- unique: true
- uppercase: true (BK-00001 format)
- match: regex `/^BK-\d{5}$/`

Format validation:

- Must start with "BK-"
- Followed by exactly 5 digits
- Examples: BK-00001, BK-12345

Field 4: price (sub-document)

Configuration:

- type: Object with nested fields
- required

Nested fields:

price.original:

- type: Number
- required: false

price.current:

- type: Number
- required

Field 5: rentalPrice

Configuration:

- type: Number
- default: 0

Field 6: lateFeePerDay

Configuration:

- type: Number
- default: 1

Field 7: totalStock

Configuration:

- type: Number
- default: 0

Field 8: availableStock

Configuration:

- type: Number
- default: 0

Field 9: status

Configuration:

- type: String
- enum: ["in-shelf", "out-of-stock"] as BookStatus[]
- default: "in-shelf"

Note: Will be auto-calculated by hook

Field 10: description

Configuration:

- type: String
- required

Index: Create text index (for search)

Field 11: cover

Configuration:

- type: String (URL)
- default: null

Validation: URL format (can use match with URL regex)

Field 12: keywords

Configuration:

- type: [String] (array)
- default: []

Index: Create text index (for search)

Purpose:

- Additional search terms
- Tags, genres, themes
- Improve search results

Field 13: category

Configuration:

- type: Schema.Types.ObjectId
- ref: “category”
- default: null

Step 2: Schema options

Add schema options:

Option	Value
`timestamps`	`true`
`toJSON`	`{ virtuals: true, versionKey: false }`
`toObject`	`{ virtuals: true, versionKey: false }`

Step 3: Create indexes

Index	Type	Purpose
`{ title: "text", description: "text", keywords: "text" }`	Text (named `"search_index"`)	Full-text search
`{ author: 1 }`	Standard	Filter by author
`{ status: 1 }`	Standard	Filter by availability
`{ category: 1 }`	Standard	Filter by category
`{ createdAt: -1 }`	Standard	New arrivals sort

Step 4: Pre-save hook for status calculation

Task: Auto-calculate status based on availableStock

Hook setup:

- Event: “save”
- Timing: “pre”
- Function: async

Logic:

Check availableStock:

- If availableStock > 0: status = "in-shelf"
- If availableStock === 0: status = "out-of-stock"

Also validate:

- availableStock must be ≤ totalStock
- If not, throw error

Why auto-calculate?

- Status always accurate
- Can't be set incorrectly
- Updates automatically when stock changes

Step 5: Pre-findOneAndUpdate hook

Task: Recalculate status when stock updated via update queries

Hook setup:

- Event: "findOneAndUpdate"
- Timing: "pre"

Implementation:

1. Get update data: `this.getUpdate()`
2. Check if availableStock was modified
3. If yes:
    - Recalculate status
    - Set in update: `this.set({ status: calculatedStatus })`

Why this hook?

- Handles: `findByIdAndUpdate()` calls
- Pre-save hook only runs on `.save()`
- Both hooks needed for complete coverage
    > This handles update queries that set `availableStock` directly. Without it, status would go stale after a `$set` update.

Step 6: Pre-delete hook

Task: When a book is deleted, all its ratings must be deleted too. Add two pre-delete hooks

---

Hook A — `pre("findOneAndDelete")` (used by `findByIdAndDelete`):

1. Get `_id` from `this.getQuery()`
2. Dynamically import `rateModel`: `const rateModel = (await import("./rate.js")).default`
3. Call `rateModel.deleteMany({ bookId })`

Hook B — `pre("deleteOne", { document: true, query: false })` (used on document instances):

1. Dynamically import `rateModel`
2. Call `rateModel.deleteMany({ bookId: this._id })`

> **Why dynamic import?** Importing `rateModel` at the top of the file would create a circular dependency (rate model imports book model). Dynamic import inside the hook avoids this.

Step 7: Create and export model

Create model:

- model: "book"
- Schema: bookSchema

Export as: bookModel

---

2. Understanding Text Search

How MongoDB text search works:

1. Indexing phase:

```
Book: "The Science of Cooking"
MongoDB:
  - Tokenizes: ["the", "science", "of", "cooking"]
  - Removes stop words: ["science", "cooking"]
  - Stems words: ["scienc", "cook"]
  - Stores in text index
```

2. Query phase:

```
Search: "cooking science"
MongoDB:
  - Tokenizes search: ["cooking", "science"]
  - Stems: ["cook", "scienc"]
  - Finds matching documents
  - Calculates relevance score
  - Returns sorted by relevance
```

3. Relevance scoring:

- More matching terms = higher score
- Title matches weighted more than description
- Exact matches scored higher

---

3. Stock Management Logic

Stock workflow:

Initial state:

```
totalStock: 10
availableStock: 10
status: "in-shelf"
```

User borrows 1 book:

```
availableStock: 9 (decremented)
totalStock: 10 (unchanged)
status: "in-shelf" (still > 0)
```

User borrows last book:

```
availableStock: 0
totalStock: 10
status: "out-of-stock" (auto-updated by hook)
```

User returns book:

```
availableStock: 1 (incremented)
totalStock: 10
status: "in-shelf" (auto-updated)
```

16.4. Rating Model

1. Create src/models/rating.ts

Task: Create Rating model with constraints

Step 1: Create rating schema

Field	Type	Required	Constraints
`ratedBy`	ObjectId, ref `"users"`	Yes	—
`bookId`	ObjectId, ref `"books"`	Yes	—
`rating`	Number	Yes	`min: 1`, `max: 5`

Step 2: Schema options

Option	Value
`timestamps`	`true`

Step 3: Unique compound index

Critical: One rating per user per book

Create index:

- Fields: `{ ratedBy: 1, bookId: 1 }`
- Options: `{ unique: true }`

What this prevents:

```
❌ User rates same book multiple times
✅ User can update their rating (same bookId + ratedBy)
```

Behavior:

- First rating: Creates new document
- Second rating: MongoDB throws duplicate key error
- Solution: Use upsert (update if exists, create if not)

Step 4: Create and export model

Collection: “ratings” Export as: ratingModel

---

2. Understanding Rating System

Rating workflow:

1. User submits rating:

```
POST /books/:bookId/ratings
{ rating: 5 }
```

2. Handler logic:

```
Try to create rating
If duplicate key error (user already rated):
  Update existing rating
Else:
  Create new rating
```

3. Recalculate book stats:

```
Get all ratings for book
Calculate average: sum / count
Update book: { avgRating, ratingCount }
```

Upsert pattern:

```
await RatingModel.findOneAndUpdate(
  { ratedBy: userId, bookId },
  { rating: newRating },
  { upsert: true, new: true },
);
```

---

17. Validations

17.1. Category Validation

Create src/validation/categories.ts

Simplest validation

categorySchema:

Field	Rules
`name`	String, min 3, max 70

Export type: CategoryInput

16.2. Author Validation

Create src/validation/author.ts

Simple validation for authors

authorSchema:

Field	Rules
`name`	String, min 1
`bio`	Optional string

authorUpdateSchema:

Same as `authorSchema` but `name` is also optional — defined separately (not via `.partial()`).

Field	Rules
`name`	Optional string, min 1
`bio`	Optional string

authorQuerySchema: For `GET /authors`.

Field	Rules
`page`	String → `transform(Number)` → `pipe(z.number().int().min(1))`, default `"1"`
`limit`	String → `transform(Number)` → `pipe(z.number().int().min(1).max(100))`, default `"10"`
`sortBy`	Enum: `"createdAt"`, `"name"` — optional, default `"createdAt"`
`sortOrder`	Enum `"asc"` or `"desc"` → transform to `1` or `-1` — optional, default `"desc"`
`search`	Optional string

Export types:

- AuthorInput
- AuthorUpdateInput
- authorQueryInput

17.3. Book Validation

Create src/validation/book.ts

Task: Create Zod schemas for book validation

Step 1: Create price sub-schema

priceSchema:

Object schema:

- original: optional number, min 0
- current: required number, min 0

Step 2: Create book schema

Field	Rules
`title`	String, min 1
`author`	`mongoDbIdSchema`
`serialNumber`	String, min 1, regex `/^BK-\d{5}$/`
`price`	Object with `original` (optional number, min 0) and `current` (required number, min 0)
`rentalPrice`	Number, min 0
`lateFeePerDay`	Number, min 0
`totalStock`	Optional number, min 0
`availableStock`	Optional number, min 0
`description`	String, min 10
`cover`	Optional, URL validation
`keywords`	Optional array of strings
`category`	Optional `mongoDbIdSchema`

> The `price` object is defined inline inside `bookSchema` — no need to export a separate `priceSchema`.

Refinement for stock:

If both stocks provided:

- availableStock ≤ totalStock
- Error: "Available stock cannot exceed total stock"

Step 3: Create update schema

bookUpdateSchema:

Make all fields of bookSchema optional:

- Use `.partial()` method

Why partial?

- Update can modify any subset of fields
- Don't need to send all fields

Step 4: Create query schema For `GET /books`. All fields come from the query string.

bookQuerySchema

Field	Rules
`page`	String → `transform(Number)` → `pipe(z.number().int().min(1))`, default `"1"`
`limit`	String → `transform(Number)` → `pipe(z.number().int().min(1).max(100))`, default `"10"`
`sortBy`	Enum: `"createdAt"`, `"title"`, `"price.current"`, `"avgRating"`, `"ratingCount"` — optional, default `"createdAt"`
`sortOrder`	Enum `"asc"` or `"desc"` → transform to `1` or `-1` — optional, default `"desc"`
`search`	Optional string
`category`	Optional `mongoDbIdSchema`
`author`	Optional `mongoDbIdSchema`
`minPrice`	String → `transform(Number)` → `pipe(z.number().min(0))` — optional
`maxPrice`	String → `transform(Number)` → `pipe(z.number().min(0))` — optional
`status`	Optional enum: `"in-shelf"` or `"out-of-stock"`
`availability`	Optional enum: `"inStock"` or `"outOfStock"`

Step 5: Create rating params schema For routes with both `:id` and `:ratingId` URL params.

bookRatingParamsSchema

Field	Rules
`id`	`mongoDbIdSchema`
`ratingId`	`mongoDbIdSchema`

Step 6: Export types

Type	From schema
`BookInput`	`bookSchema`
`BookUpdateInput`	`bookUpdateSchema`
`BookQueryInput`	`bookQuerySchema`

17.4. Rating Validation

Create src/validation/rate.ts

ratingSchema:

Single field:

Field	Rules
`rating`	Number, min 1, max 5

Export type: RatingInput

---

18. Handlers (controllers)

18.1. Category handler

Create src/handlers/categories.ts

Simplest handlers - standard CRUD

Handler 1: getCategories (GET /categories)

Steps:

1. Query all categories
2. Sort by name (alphabetical)
3. Return array

No pagination:

- Categories are limited (usually < 50)
- Returning all is acceptable

Handler 2: getCategoryById (GET /categories/:id)

Standard find by ID

Handler 3: createCategory (POST /categories) - Admin

Steps:

1. Extract category data
2. Create category
3. Return created with 201

Handler 4: updateCategory (PUT /categories/:id) - Admin

Standard update with validation

1. Extract update data and id
2. Find category by ID and update it
3. If not found → return `404`
4. Return `200` with updated category

Handler 5: deleteCategory (DELETE /categories/:id) - Admin

Before deleting, check if any books reference this category.

Steps:

1. Count books with this category `bookModel.countDocuments({ category: req.params.id })`
2. If count > 0: throw error or set category to null
3. Delete category — if not found return `404`
4. Return `200` with deleted category

18.2. Author handler

1. Create src/handlers/author.ts

Handler 1: getAuthors (GET /authors)

Steps:

Reads from `req.parsedQuery`.

1. Destructure `limit`, `page`, `sortBy`, `sortOrder`, `search` from `parsedQuery` with defaults
2. Build filter: if `search` → `{ name: { $regex: search, $options: "i" } }`, otherwise `{}`
3. Run in parallel: `find(filter).populate("books", "title cover").limit().skip().sort()`, and `countDocuments(filter)`
4. Return `200` with `data` and `pagination`

Name search pattern:

```
const filter = search ? { name: { $regex: search, $options: "i" } } : {};
```

$regex options:

- "i" = case-insensitive
- Allows partial matches

Handler 2: getAuthorById (GET /authors/:id)

Steps:

1. `findById(req.params.id).populate("books")`
2. If not found → return `404`
3. Return `200` with author (includes all book data via virtual)

Handler 3: createAuthor (POST /authors) - Admin

Steps:

1. `authorModel.create(req.body)`
2. Return `201` with created author
3. On error code `11000` → return `409` "Author already exists"
4. On other errors → return `400` with error message

Handler 4: updateAuthor (PUT /authors/:id) - Admin

Steps:

1. `findByIdAndUpdate(req.params.id, { $set: req.body }, { new: true })`
2. If not found → return `404`
3. Return `200` with updated author

Handler 5: deleteAuthor (DELETE /authors/:id) - Admin

Critical: Prevent deletion if author has books!

Steps:

Before deleting, check if the author has books via the virtual.

1. `findById(req.params.id).populate("books")`
2. If not found → return `404`
3. Cast author to access `books`: `const authorWithBooks = author as typeof author & { books?: unknown[] }`
4. If `authorWithBooks.books?.length > 0` → return `400` "Cannot delete author with existing books. Please reassign or delete the books first."
5. `authorModel.findByIdAndDelete(req.params.id)`
6. Return `200` "Author deleted successfully"

> Checking via the virtual (not a `countDocuments` query) avoids an extra DB call — the populate in step 1 already fetches the books.

Why prevent deletion?

- Books reference this author
- Deleting would break referential integrity
- User must reassign or delete books first

18.3. Book handler

1 Understanding Book Handler Complexity

Book handlers are the most complex because:

1. Multiple query types (search, filter, sort)
2. Pagination
3. Population (author, category)
4. Text search with relevance scoring
5. Stock management
6. Rating system integration
7. Recommendations algorithm

---

2. Create src/handlers/books.ts

Task: Create comprehensive book handlers

Handler 1: createBook (POST /books) - Admin

Steps:

1. Extract book data from req.body
2. Create book: `bookModel.create(data)`
3. Populate author and category on created book
4. Return created book with 201 status

Error handling:

- Duplicate serialNumber: MongoDB throws code 11000
- Error handler catches and formats

Handler 2: getBooks (GET /books)

Purpose: Paginated list with filtering, search, and sorting

Steps:

Part A: Extract query parameters

From req.parsedQuery:

- page, limit (from pagination)
- search (optional text search)
- category (optional category filter)
- author (optional author filter)
- minPrice, maxPrice (optional price range)
- status (optional availability filter)
- sortBy (default "createdAt")
- sortOrder (default -1, newest first)

Part B: Build filter object

Create empty filter object: `{}`

Add filters conditionally:

If search exists:

- Add: `filter.$text = { $search: search }`

If category exists:

- Add: `filter.category = new Types.ObjectId(category)`

If author exists:

- Add: `filter.author = new Types.ObjectId(author)`

If minPrice or maxPrice:

- Add: `filter["price.current"] = { $gte: minPrice, $lte: maxPrice }`

If status:

- Add: `filter.status = status`

Add :

- `availability === "inStock"` → `filter.availableStock = { $gt: 0 }`
  - `availability === "outOfStock"` → `filter.availableStock = 0`

Part C: Build sort object

Create sort object based on sortBy and sortOrder

Part D: Execute parallel queries

Use Promise.all for:

Query 1: Find books

- Apply filter
- Apply sort
- Skip and limit (pagination)
- Populate: author (name)
- Populate: category (name)
- If text search: project text score

Query 2: Count total

- Same filter
- countDocuments()

Part E: Build response

Return:

- success: true
- data: books array
- pagination: { total, page, limit, totalPages }

Why parallel queries?

```
// ❌ Sequential (slow)
const books = await query1;   // 100ms
const total = await query2;   // 50ms
// Total: 150ms

// ✅ Parallel (fast)
const [books, total] = await Promise.all([query1, query2]);
// Total: 100ms (max of both)
```

Handler 3: getBookById (GET /books/:id)

Steps:

1. Extract ID from req.params.id
2. Query: findById with populate (author, category)
3. If not found: throw 404 error
4. Return book data with 200

Handler 4: updateBook (PUT /books/:id) - Admin

Steps:

1. Extract ID and updates
2. Update: `findByIdAndUpdate(id, { $set: updates }, { new: true})`
3. Populate author and category
4. If not found: throw 404 error
5. Return updated book

Why $set?

- Only updates provided fields
- Doesn't overwrite entire document

Handler 5: deleteBook (DELETE /books/:id) - Admin Before deleting, check for active borrows.

Steps:

1. Extract ID
2. Count borrows: `{ book: id, status: { $in: ["pending", "active", "overdue"] } }`
3. If count `> 0` → return `400` "Cannot delete book with N active borrow(s)"
4. Delete: `findByIdAndDelete(id)`
5. If not found: throw 404 error
6. Return success message

> The cascade hook on the model will automatically delete all ratings after the book is deleted.

Handler 6: getBookRatings (GET /books/:id/ratings)

Steps:

1. Extract book ID
2. Verify book exists
3. Query ratings: `ratingModel.find({ bookId })`
4. Populate: ratedBy (firstName, lastName)
5. Sort by `createdAt: -1`
6. Return ratings array

Handler 7: addBookRating (POST /books/:id/ratings) - Auth

Steps:

1. Get `bookId` from `req.params.id`, `rating` from `req.body`, `userId` from `req.user._id`
2. Check eligibility: `borrowModel.exists({ user: userId, book: bookId, status: { $in: ["returned", "active", "overdue"] } })` — if not found return `403` "You can only rate books that you have borrowed or purchased"
3. Check for existing rating: `rateModel.findOne({ ratedBy: userId, bookId })`
4. If exists → update it: `findByIdAndUpdate(existingRating._id, { $set: { rating } }, { new: true })`
5. If not exists → create: `rateModel.create({ ratedBy: userId, bookId, rating })`
6. Recalculate book stats: fetch all ratings for the book, compute average, update book with `$set: { avgRating, ratingCount }`
7. Return `200` with the rating document — message: `"Rating updated"` or `"Rating added"` depending on which path ran
8. On error code `11000` → return `409` "You have already rated this book"

Handler 8: deleteBookRating (DELETE /books/:id/ratings/:ratingId) - Auth

Steps:

Part A: Find rating

Query by ratingId

Part B: Verify ownership

Check:

- If `rating.ratedBy.toString() !== userId.toString()`
- user is not admin
- If neither: throw 403 error

Part C: Delete rating

Use deleteOne

Part D: Recalculate average

fetch remaining ratings for `bookId`, compute average (return `0` if none left), update book `avgRating` and `ratingCount`

Part E: Return success Return `200` "Rating deleted successfully"

Handler 9: getRecommendations (GET /books/recommendations) - Auth

Purpose: Personalized book suggestions

Algorithm:

Step 1: Get user's favorite books

Extract from: req.user.books.favorites

Step 2: Find categories of favorites

Query favorites to get their categories Extract unique category IDs

Step 3: Find similar books

Query:

- Category in user's favorite categories
- Exclude books user already has
- Sort by avgRating DESC
- Limit 10

Step 4: Fallback if no favorites

If user has no favorites:

- Return popular books (high ratings)

Step 5: Return recommendations

Handler 10: getPopularBooks (GET /books/popular)

Steps:

1. Parse `limit` from `req.query` (default `10`)
2. Find books where `status: "in-shelf"`
3. Populate author (name) and category (name)
4. Sort by `ratingCount: -1, avgRating: -1`
5. Limit and return `200`

Handler 11: getNewArrivals (GET /books/new)

Steps:

1. Parse `limit` from `req.query` (default `10`)
2. Find books where `status: "in-shelf"`
3. Populate author (name) and category (name)
4. Sort by `createdAt: -1`
5. Limit and return `200`

---

19. Rating System Implementation

19.1 Complete Rating Flow

User rates a book:

```
1. User interaction:
   ┌──────────────────────────┐
   │ POST /books/123/ratings  │
   │ { rating: 5 }            │
   └──────────────────────────┘
                ↓
2. Middleware:
   ┌──────────────────────────┐
   │ CheckAuth: Verify token  │
   │ Attach user to request   │
   └──────────────────────────┘
                ↓
3. Handler:
   ┌──────────────────────────┐
   │ Verify user borrowed book │
   │ Upsert rating            │
   │ Recalculate book average │
   │ Update book document     │
   └──────────────────────────┘
                ↓
4. Response:
   ┌──────────────────────────┐
   │ {                        │
   │   success: true,         │
   │   data: {                │
   │     rating: 5,           │
   │     ratedBy: {...}       │
   │   }                      │
   │ }                        │
   └──────────────────────────┘
```

19.2 Rating Statistics Update

When to recalculate:

- After adding rating
- After updating rating
- After deleting rating

Calculation method:

```
const ratings = await ratingModel.find({ bookId });
const sum = ratings.reduce((acc, r) => acc + r.rating, 0);
const avg = ratings.length ? sum / ratings.length : 0;

await bookModel.findByIdAndUpdate(bookId, {
  avgRating: avg,
  ratingCount: ratings.length,
});
```

---

20. Routes

20.1 Category Routes

Prefix: `/categories`

create src/routers/category.ts

Method	Path	Middleware	Handler	Access
GET	`/`	—	`getCategories`	Public
GET	`/:id`	`validateParamsSchema(idParamsSchema)`	`getCategoryById`	Public
POST	`/`	`CheckAuth`, `isAdmin`, `validateBodySchema(categorySchema)`	`createCategory`	Admin
PUT	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(categorySchema)`	`updateCategory`	Admin
DELETE	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`	`deleteCategory`	Admin

Mount in `app.ts`: `app.use("/categories", categoryRouter)`

20.2 Author Routes

Prefix: `/authors`

create src/routers/author.ts

Method	Path	Middleware	Handler	Access
GET	`/`	`validateQuerySchema(authorQuerySchema)`	`getAuthors`	Public
GET	`/:id`	`validateParamsSchema(idParamsSchema)`	`getAuthorById`	Public
POST	`/`	`CheckAuth`, `isAdmin`, `validateBodySchema(authorSchema)`	`createAuthor`	Admin
PUT	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(authorUpdateSchema)`	`updateAuthor`	Admin
DELETE	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`	`deleteAuthor`	Admin

> `GET /` uses `authorQuerySchema`, not a generic pagination schema — it includes `search` and `sortBy`.

> PUT uses `authorUpdateSchema` (name optional), not `authorSchema` (name required).

Mount in `app.ts`: `app.use("/authors", authorRouter)`

20.3 Book Routes

Prefix: `/books`

create src/routers/book.ts Register routes in this exact order. All static named routes must come before `/:id`.

Public:

#	Method	Path	Middleware
1	GET	`/`	`validateQuerySchema(bookQuerySchema)`
2	GET	`/search`	—
3	GET	`/popular`	—
4	GET	`/new`	—
5	GET	`/category/:categoryId`	—
6	GET	`/author/:authorId`	—
7	GET	`/recommendations`	`CheckAuth`
8	GET	`/:id`	`validateParamsSchema(idParamsSchema)`
9	GET	`/:id/ratings`	`validateParamsSchema(idParamsSchema)`

Protected:

#	Method	Path	Middleware
10	POST	`/:id/ratings`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(ratingSchema)`
11	DELETE	`/:id/ratings/:ratingId`	`CheckAuth`, `validateParamsSchema(bookRatingParamsSchema)`

Admin:

#	Method	Path	Middleware
12	POST	`/`	`CheckAuth`, `isAdmin`, `validateBodySchema(bookSchema)`
13	PUT	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(bookUpdateSchema)`
14	DELETE	`/:id`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`

> Named routes (`/search`, `/popular`, `/new`, `/recommendations`) MUST be defined BEFORE `/:id`.

> `/recommendations` (route 7) must be before `/:id` (route 8) — otherwise Express matches "recommendations" as an ID.

> `DELETE /:id/ratings/:ratingId` uses `bookRatingParamsSchema` (validates both `:id` and `:ratingId`), not `idParamsSchema`.

> `GET /` uses `bookQuerySchema`, not a generic `paginationSchema` — the book query has many more fields.

Mount in `app.ts`: `app.use("/books", bookRouter)`

---

21. Borrowings management

21.1 Borrow Types

1. Understanding Borrow Status Flow

Borrow lifecycle has multiple states: A borrow does not go directly from "requested" to "active". It moves through states:

```
User submits request
        ↓
    pending  ──→  rejected (admin denies)
        ↓
    active   ──→  cancelled (user cancels, only from pending)
        ↓
   returned  ←── overdue (scheduler marks as overdue, still returnable)
```

Status definitions:

Status	Meaning	Who Can Set
pending	Awaiting admin approval	System (on request)
active	Book borrowed and in use	admin (on borrow)
rejected	Admin denied request	Admin
overdue	Past due date, not returned	Scheduler (automated)
returned	Book returned	User/Admin
cancelled	Request cancelled before borrow	User

---

2. Create types/models/borrow.ts

Task: Define borrow types

Type 1: BorrowStatus

Union type with all statuses:

```
"pending" | "rejected" | "active" | "overdue" | "returned" | "cancelled";
```

Add to types/common.ts (with other status types)

Type 2: IBorrow interface

> Don't extend `BaseDocument` — Mongoose adds `_id`, `createdAt`, `updatedAt` automatically via schema options.

Core fields:

Field	Type	Optional	Notes
user	Types.ObjectId	No	Who borrowed
book	Types.ObjectId	No	Which book
borrowDate	Date	No	When borrowed
dueDate	Date	No	When it must be returned
returnDate	Date	Yes (?)	Set when returned
status	BorrowStatus	No	Current lifecycle state
comments	string	Yes (?)	User-provided notes
approvedBy	Types.ObjectId	Yes (?)	Admin who approved
approvedAt	Date	Yes (?)	When approved
returnedAt	Date	Yes (?)	Timestamp of return
rejectionReason	string	Yes (?)	Reason if rejected

Type 3: IBorrowWithVirtuals

Extends: IBorrow

Add virtual fields (calculated, not stored):

- lateFee: number
- daysOverdue: number

These are computed on-the-fly based on current date and dueDate

---

21.2. Date Helper Functions

Pure functions, no database calls, no side effects. They will be used by the virtual fields and handlers.

1. Why Separate Date Helpers?

Date calculations are reused throughout:

- Calculate due dates
- Check if overdue
- Calculate days difference
- Add/subtract days

Better to centralize than duplicate

---

2. Create src/utils/dateHelpers.ts

Task: Create date utility functions

Function 1: addDays

Purpose: Add days to a date, Create a **new** Date from the input (never mutate the original)

Signature:

```
addDays(date: Date, days: number): Date
```

Implementation:

1. Create new Date from input (don't mutate)
2. Get current date value: `date.getDate()`
3. Add days: `setDate(date.getDate() + days)`
4. Return new date

Usage:

```
const today = new Date();
const nextWeek = addDays(today, 7);
```

Function 2: daysDifference

Purpose: Calculate days between two dates

Signature:

```
daysDifference(date1: Date, date2: Date): number
```

Implementation:

1. Get time in milliseconds for both
2. Calculate difference: `Math.abs(time1 - time2)`
3. Convert to whole days: divide by `1000 * 60 * 60 * 24`
4. Round down: `Math.floor()`
5. Return the result

Usage:

```
const start = new Date("2024-01-01");
const end = new Date("2024-01-08");
const days = daysDifference(start, end); // 7
```

Function 3: daysOverdue

Purpose: Calculate how many days past due date

Signature:

```
daysOverdue(dueDate: Date): number
```

Implementation:

1. Get current date
2. If current date <= due date: return 0 (not overdue)
3. Otherwise: calculate difference (current - due)
4. Convert to days and return

Usage:

```
const due = new Date("2024-01-01");
const overdue = daysOverdue(due); // e.g., 5 if today is Jan 6
```

21.3. Borrow Model with Virtual Fields

1. Understanding the Challenge

Borrow model is complex because:

1. Multiple status states with transitions
2. Date-based calculations (overdue, fees)
3. Virtual fields (computed, not stored)
4. Stock integration (borrow affects availability)
5. Compound indexes for performance
6. Business rules enforcement

---

2. Create src/models/borrow.ts

Task: Create Borrow model with virtual fields

Step 1 — Define the schema fields

Field Definitions:

Field	Mongoose type	Required	Default	Constraints
user	Schema.Types.ObjectId, ref "user"	Yes	—	—
book	Schema.Types.ObjectId, ref "book"	Yes	—	—
borrowDate	Date	Yes	Date.now	—
dueDate	Date	Yes	—	—
returnDate	Date	No	null	Set on return
status	String, enum of BorrowStatus values	No	"pending"	—
comments	String	No	null	trim: true
approvedBy	Schema.Types.ObjectId, ref "user"	No	null	—
approvedAt	Date	No	null	—
returnedAt	Date	No	null	—
rejectionReason	String	No	null	—

Step 2: Schema options

Standard options:

- timestamps: true
- toJSON: { virtuals: true, versionKey: false }
- toObject: { virtuals: true, versionKey: false }

Critical: without the `virtuals: true` on both `toJSON` and `toObject`, `lateFee` and `daysOverdue` will silently disappear from every API response.

Step 3: Create indexes

Create four indexes. Add them after the schema definition using `schema.index()`.

Indexes Definition

Index fields	Options	Purpose
`{ user: 1, status: 1 }`	—	Find a user's borrows by status
`{ book: 1, status: 1 }`	—	Check if a book is currently borrowed
`{ status: 1, dueDate: 1 }`	—	Find overdue borrows efficiently
`{ user: 1, book: 1, status: 1 }`	`unique: true`, `partialFilterExpression: { status: "active" }`	Prevent duplicate active borrows

> The partial unique index prevents a user from having two active borrows for the same book. Because of `partialFilterExpression`, uniqueness only applies when `status` is `"active"` — the same user can borrow the same book again after returning it.

Step 4: Virtual field - lateFee

Task: Calculate late fee on-the-fly

> Always use a regular function, not an arrow function — `this` must refer to the document. **Virtual configuration:**

Name: "lateFee"

Getter function logic:

1. Check if applicable:
    - Status must be: “active”, “overdue”, or “returned”
    - If not, return 0
2. Get book data (need lateFeePerDay)
    - Populate or fetch book
    - If not available, return 0
3. Determine comparison date:
    - If returned: use returnedAt
    - If not: use current date
4. Calculate late fee:
    - Use calculateLateFee helper
    - Pass: dueDate, lateFeePerDay, comparison date
5. Return calculated fee

Implementation pattern:

```
schema.virtual("lateFee").get(function () {
  // 1. Check if applicable
  const applicableStatuses = ["active", "overdue", "returned"];
  if (!applicableStatuses.includes(this.status)) {
    return 0;
  }

  // 2. Get book (if populated)
  // this.book could be ObjectId or populated object

  // 3. Determine date
  const compareDate = this.returnedAt || new Date();

  // 4. Calculate
  // Use helper function

  // 5. Return
});
```

Note: Virtual can’t be async, so book must be populated beforehand

---

Step 6: Virtual field - daysOverdue Simpler than lateFee (no book needed)

Task: Calculate days overdue

Virtual configuration:

Name: “daysOverdue”

Getter function logic:

1. Check if applicable (same statuses)
2. Determine comparison date
3. Use daysOverdue helper
4. Return overdue days

Step 7: Create and export model

Model: “borrow” Export as: borrowModel

21.3. Understanding Virtual Fields

Virtual fields are calculated at read time, never stored. They are always accurate because they recalculate from the current date on every read.

1. Virtual vs Regular Fields

Regular field (stored):

```
lateFee: {
  type: Number,
  default: 0
}
// Stored in MongoDB
// Value becomes stale (needs updates)
```

Virtual field (calculated):

```
schema.virtual("lateFee").get(function() {
  return calculateLateFee(this.dueDate, ...);
});
// NOT stored in MongoDB
// Always current (calculated on read)
```

---

2. Why Virtual Fields for Fees?

Storing it would require a daily job to update every overdue record — and the value would still be stale between runs.

With a virtual:

Day	What happens
Day 1	Calculates `0` ✅
Day 8 (due date)	Calculates `0` ✅
Day 9	Calculates fee × 1 ✅
Day 10	Calculates fee × 2 ✅

No updates needed. Always accurate.

Limitations

Limitation	Workaround
Cannot query by virtual	Query by real stored fields (`status` , `dueDate`) instead
Cannot sort by virtual in MongoDB	Fetch first, then sort in JavaScript
Cannot be async	Populate the book before reading the virtual — getters run synchronously

Virtual vs stored field — when to use which

Use virtual	Use stored field
Value changes frequently	Value is set once and rarely changes
Calculated from existing document data	Independent data
No need to `$match` or `$sort` by it in MongoDB	Need to query or sort by it

---

21.4. Borrow Validation

Create src/validation/borrow.ts

Task: Create borrow validation schemas

Define `BORROW_STATUS` as a `const` tuple at the top of the file — reused in the query schema:

```
const BORROW_STATUS = ["pending", "approved", "rejected", "active", "returned", "overdue", "cancelled"] as const
```

Schema 1: borrowSchema

Purpose: Create borrow request

Fields:

bookId:

- Use mongoIdSchema
- Required

dueDate:

- Type: date string (ISO)
- Required
- Transform: Convert to Date object
- Validate: Must be in future

Transformation pattern:

```
dueDate: z.string()
  .transform((val) => new Date(val))
  .refine((date) => date > new Date(), "Due date must be in the future");
```

comments:

- Optional string
- Max length: 500

Why date as string in validation?

Request body:

```
{
  "bookId": "...",
  "dueDate": "2024-12-31T00:00:00.000Z"
}
```

> JSON doesn't have Date type, so it comes as string. Zod transforms to Date.

> `dueDate` arrives as a string from the JSON body. The transform converts it; the refine validates the resulting Date.

> `borrowDate` and `returnDate` are **not** accepted from the client — they are server-controlled.

Schema 2: extendBorrowSchema

Fields:

Field	Rules
`additionalDays`	Number, integer, min 1, max 30, optional, default `7`

Schema 3: rejectBorrowSchema

Fields:

Field	Rules
`reason`	Optional, string, max 500 chars

Schema 4: `borrowQuerySchema` — query of `GET /` (admin)

Schema 4: borrowQuerySchema

More detailed than a standard pagination schema — includes borrow-specific filters and sort options.

Fields:

Field	Rules	Notes
`page`	String → transform to number, min 1, default `"1"`	Use `.transform(Number).pipe(z.number().int().min(1))`
`limit`	String → transform to number, min 1, max 100, default `"10"`	Same pattern
`sortBy`	Enum: `"createdAt"`, `"dueDate"`, `"borrowDate"`, `"status"` — optional, default `"createdAt"`	Only these 4 values are valid
`sortOrder`	Enum: `"asc"` or `"desc"` → transform to `1` or `-1` — optional, default `"desc"`	Transform to Mongoose sort direction
`status`	`z.enum(BORROW_STATUS)`, optional	Filter by status
`user`	`mongoDbIdSchema`, optional	Filter by user ID
`book`	`mongoDbIdSchema`, optional	Filter by book ID

> Using `z.enum` for `sortBy` means the validated value is safe to pass directly to Mongoose's `.sort()` — no risk of injection from user input.

Type exports

Type name	From schema
`BorrowInput`	`borrowSchema`
`ExtendBorrowInput`	`extendBorrowSchema`
`RejectBorrowInput`	`rejectBorrowSchema`
`BorrowQueryInput`	`borrowQuerySchema`

21.5. Borrow Handlers

1. Understanding Handler Complexity

Borrow handlers involve:

1. Status transitions (complex state machine)
2. Stock management (increment/decrement)
3. User data updates (borrowed list)
4. Business rules (borrow limit, eligibility)
5. Virtual field population
6. Date calculations

---

2. Create src/handlers/borrow.ts

Task: Create comprehensive borrow handlers

Handler 1: requestBorrow (POST /borrows/request)

Purpose: User requests to borrow a book

Access: Protected (authenticated user)

Steps:

1. Get `bookId`, `dueDate`, `comments` from `req.body` — get `userId` from `req.user._id`
2. Find the book — if not found return `404`
3. If `book.availableStock <= 0` → return `400` "Book is not available for borrowing"
4. Check for an existing borrow: `{ user: userId, book: bookId, status: { $in: ["pending", "approved", "active"] } }` — if found return `409` "You already have an active borrow for this book"
5. Count current borrows: `{ user: userId, status: { $in: ["pending", "approved", "active"] } }` — if `>= 5` return `400` "You have reached your borrow limit (5 books)"
6. Create borrow with `status: "pending"` — include `comments` only if provided
7. Populate `user` (firstName, lastName, email) and `book` (title, cover, author)
8. Return `201`

Why check existing active borrow?

- Prevents borrowing same book twice
- Different from unique index (which is per active status)
- Better error message to user

Handler 2: approveBorrow (POST /borrows/:id/approve)

Purpose: Admin approves borrow request

Access: Admin only

Steps:

1. Get `id` from params, `adminId` from `req.user._id`
2. Find borrow — if not found return `404`
3. If `status !== "pending"` → return `400` "Only pending requests can be approved"
4. Atomically decrement stock: `findOneAndUpdate({ _id: borrow.book, availableStock: { $gt: 0 } }, { $inc: { availableStock: -1 } }, { new: true })` — if result is `null` return `400` "Book is no longer available for borrowing" **Why $inc?**

- Atomic operation (safe for concurrent requests)
- Mongoose pre-save hook will recalculate status

5. Set `borrow.status = "active"`, `borrow.approvedAt = new Date()`, `borrow.approvedBy = adminId` — save
6. Add book to user's borrowed list: `$addToSet: { "books.borrowed": borrow.book }`
7. Populate and return `200`

> **Why use atomic `findOneAndUpdate` with `{ $gt: 0 }`?** Checking stock then updating are two separate operations — another request could decrement to zero between them. This single atomic operation either decrements or returns `null` if already out of stock. No race condition.

> Status becomes `"active"` (not `"approved"`) — the book is now in use.

Handler 3: rejectBorrow (POST /borrows/:id/reject)

Purpose: Admin rejects borrow request

Access: Admin only

Steps:

1. Find borrow — if not found return `404`
2. If `status !== "pending"` → return `400` "Only pending requests can be rejected"
3. Get `reason` from `req.body`
4. Set `status = "rejected"` — if `reason` provided, set `rejectionReason`
5. Save, populate (book title/cover, user firstName/lastName/email), return `200`

> No stock changes — stock is only decremented on approval.

Handler 4: returnBook (POST /borrows/:id/return)

Purpose: User returns book

Access: Protected (owner or admin)

Steps:

1. Get `id` from params, `userId` from `req.user._id`, determine `isAdmin` via `req.user.role === "admin"`
2. Find borrow — if not found return `404`
3. If not admin and `borrow.user.toString() !== userId.toString()` → return `403`
4. If `status` not `"approved"` and not `"active"` → return `400` "Only active borrows can be returned"
5. **Save raw IDs before any populate:** `const rawBookId = borrow.book` and `const rawUserId = borrow.user`
6. Set `status = "returned"`, set both `returnDate` and `returnedAt` to `new Date()` — save
7. Populate `book` with `lateFeePerDay` — then read `const lateFee = populatedBorrow.lateFee || 0`
8. Restore book stock: `findByIdAndUpdate(rawBookId, { $inc: { availableStock: 1 } })`
9. Update user book lists using `rawUserId`: `$pull: { "books.borrowed": rawBookId }` and `$addToSet: { "books.read": rawBookId }`
10. Return `200` with borrow as `.toObject()`, `lateFee`, and `isOverdue: lateFee > 0`

> **Why save raw IDs before populate?** `.populate()` replaces ObjectId references with full objects. Using those mutated fields in `$pull` or `$addToSet` breaks the match. Save the raw ObjectIds first, use them for all updates.

Handler 5: cancelBorrow (POST /borrows/:id/cancel)

Purpose: User cancels pending request

Access: Protected (owner only)

Steps:

1. Find borrow — if not found return `404`
2. If `borrow.user.toString() !== userId.toString()` → return `403`
3. If `status !== "pending"` → return `400` "Only pending requests can be cancelled"
4. Set `status = "cancelled"` — save and return `200`

> No stock changes — stock is never touched for a pending borrow.

Handler 6: extendBorrow (POST /borrows/:id/extend)

Purpose: Extend due date

Access: Protected (owner only)

Steps:

1. Get `additionalDays` from `req.body` (default `7`), cap it: `Math.min(Math.max(additionalDays, 1), 30)`
2. Find borrow — if not found return `404`
3. If not owner → return `403`
4. If `status` not `"approved"` and not `"active"` → return `400`
5. If `borrow.dueDate < new Date()` → return `400` "Cannot extend overdue borrows. Please return the book first."
6. Set `borrow.dueDate = addDays(borrow.dueDate, cappedDays)` — save
7. Return `200` with `{ newDueDate: borrow.dueDate, additionalDays: cappedDays }`

Handler 7: getBorrows (GET /borrows)

Purpose: Admin views all borrows Reads from `req.parsedQuery` (set by `validateQuerySchema(borrowQuerySchema)`), not `req.query`. **Access:** Admin only

---

Steps:

1. Destructure from `parsedQuery`: `limit`, `page`, `sortBy`, `sortOrder`, `status`, `user`, `book` — with defaults (`limit = 10`, `page = 1`, `sortBy = "createdAt"`, `sortOrder = -1`)
2. Build filter — add `status`, `user`, `book` only if provided
3. Run in parallel: `find` with populate + pagination + sort, and `countDocuments`
   - Populate `user`: firstName, lastName, email, avatar
   - Populate `book`: title, cover, price
4. Return `200` with `data` and `pagination` (total, page, limit, totalPages)

Handler 8: getUserBorrows (GET /borrows/my)

Purpose: User views their borrows Reads from `req.query` directly (no query validation middleware on this route) — parse manually. **Access:** Protected

---

Steps:

1. Get `userId` from `req.user._id`
2. Read `status`, `limit`, `page` from `req.query` as strings
3. Parse: `limit = Math.min(parseInt(limit) || 10, 50)` and `page = Math.max(parseInt(page) || 1, 1)`
4. Build filter `{ user: userId }` — add `status` if provided
5. Run in parallel: `find` populated with book (title, cover, author, price), sorted by `createdAt: -1`, and `countDocuments`
6. Return `200` with paginated response

Handler 9: getBorrowById (GET /borrows/:id)

Purpose: Get single borrow details

Access: Protected (owner or admin)

Steps:

1. Find borrow with `.populate("book", "title cover author price")` and `.populate("user", "firstName lastName email avatar")` — if not found return `404`
2. Extract user ID from populated document: `(borrow.user as { _id?: unknown })._id || borrow.user`
3. If not admin and IDs don't match → return `403` "Access denied"
4. Return `200` — virtuals are included automatically from schema options

> After populate, `borrow.user` is an object, not a plain ObjectId. Extract `._id` from it before comparing to `req.user._id`.

Handler 10: getOverdueBorrows (GET /borrows/overdue)

Purpose: Admin gets overdue list

Access: Admin only

Steps:

1. Query with both conditions:
   - `status: { $in: ["active", "overdue"] }` — both because the status may not have updated yet
   - `dueDate: { $lt: new Date() }` — due date has passed
2. Populate `user` (firstName, lastName, email) and `book` (title, cover)
3. Sort by `dueDate: 1` — most overdue borrows appear first
4. Return `200` with `data` and `count: borrows.length`

---

9. Borrow Routes

File: `src/routers/borrow.ts` | **Prefix:** `/borrows`

Register routes in this exact order. Express matches top-to-bottom.

User routes

#	Method	Path	Middleware
1	GET	`/my`	`CheckAuth`
2	POST	`/request`	`CheckAuth`, `validateBodySchema(borrowSchema)`
3	POST	`/:id/cancel`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`
4	POST	`/:id/extend`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(extendBorrowSchema)`
5	POST	`/:id/return`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`

Admin routes

#	Method	Path	Middleware
6	GET	`/`	`CheckAuth`, `isAdmin`, `validateQuerySchema(borrowQuerySchema)`
7	GET	`/overdue`	`CheckAuth`, `isAdmin`
8	GET	`/:id`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`
9	POST	`/:id/approve`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`
10	POST	`/:id/reject`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(rejectBorrowSchema)`

> `GET /overdue` must be before `GET /:id` — otherwise Express treats "overdue" as an ID and calls `getBorrowById` instead. Add a comment on that route: `NOTE: Must be registered BEFORE /:id`.

> `GET /` is safe before `GET /overdue` — both are exact paths with no parameters. Only `/:id` captures named strings.

Mount in `app.ts`: `app.use("/borrows", borrowRouter)`

---

22. Order Management

22.1 Order Types

1. Add to `types/common.ts`

Add these union types:

Type	Values
`OrderStatus`	`"pending"`, `"processing"`, `"shipped"`, `"delivered"`, `"cancelled"`, `"completed"`
`PaymentStatus`	`"pending"`, `"paid"`, `"failed"`, `"refunded"`
`DeliveryMethod`	`"pickup"`, `"delivery"`
`PaymentMethod`	`"cash_on_delivery"`

2. create `src/types/models/order.ts` file

> Do not extend `BaseDocument` — `IOrder` is a plain interface. Mongoose adds `_id`, `createdAt`, `updatedAt` via schema options.

3. create `ShippingAddress` interface

Field	Type	Required
`fullName`	`string`	No (`?`)
`street`	`string`	Yes
`city`	`string`	Yes
`state`	`string`	No (`?`)
`postalCode`	`string`	Yes
`country`	`string`	Yes
`phone`	`string`	Yes
`instructions`	`string`	No (`?`)

4. create `OrderBookItem` interface

Stores a snapshot of book data at purchase time — not just a reference.

Field	Type	Notes
`bookId`	`Types.ObjectId`	Ref to book
`title`	`string`	Copied from book at purchase
`quantity`	`number`	—
`price`	`number`	Price at time of order

> **Why snapshot?** If a book's price changes tomorrow, the order must still show what the customer paid today. Copying `title` and `price` directly prevents this drift.

5. create `StatusHistoryEntry` interface

Field	Type	Required
`status`	`string`	Yes
`changedAt`	`Date`	Yes
`changedBy`	`Types.ObjectId`	No (`?`)
`note`	`string`	No (`?`)

6. create `IOrder` interface

Field	Type	Required	Notes
`userId`	`Types.ObjectId`	Yes	Ref to users
`books`	`OrderBookItem[]`	Yes	Min 1 item
`subtotal`	`number`	Yes	Before shipping
`shippingFee`	`number`	Yes	0 for pickup
`totalPrice`	`number`	Yes	subtotal + shippingFee
`deliveryMethod`	`DeliveryMethod`	Yes	—
`shippingAddress`	`ShippingAddress`	No (`?`)	Required if delivery
`paymentMethod`	`PaymentMethod`	Yes	—
`paymentStatus`	`PaymentStatus`	Yes	—
`status`	`OrderStatus`	Yes	—
`orderNotes`	`string`	No (`?`)	Max 500
`trackingNumber`	`string`	No (`?`)	—
`cancellationReason`	`string`	No (`?`)	—
`cancelledAt`	`Date`	No (`?`)	—
`statusHistory`	`StatusHistoryEntry[]`	Yes	Full audit trail

7. create `IOrderDocument` type

```
export type IOrderDocument = Document<Types.ObjectId> & IOrder;
```

Imports: `Types`, `Document` from `"mongoose"` — union types from `"../common.js"`

22.2. Understanding MongoDB Transactions

1. The problem without transactions

When creating an order with 2 books:

1. Decrement Book A stock ✅
2. Decrement Book B stock ✅
3. Create order document ❌ (throws error)

Steps 1 and 2 already ran — stock is decremented but no order exists. Data is now inconsistent.

2. The solution

A transaction wraps all operations into one atomic unit — either everything succeeds, or everything is rolled back.

3. Transaction sequence

1. `const session = await mongoose.startSession()`
2. `session.startTransaction()`
3. Run all queries, passing `{ session }` to each one
4. `await session.commitTransaction()` — on success
5. `await session.abortTransaction()` — on error
6. `session.endSession()` — always, in a `finally` block

> **Critical:** Every Mongoose query inside a transaction MUST receive `{ session }`. Forgetting it on even one query means that operation runs outside the transaction and won't be rolled back.

> **Replica set requirement:** Transactions only work on MongoDB replica sets. MongoDB Atlas always supports them. For local MongoDB, add `--replSet rs0` to your startup command and run `rs.initiate()` once.

22.3. Order Model

File: `src/models/order.ts`

Field 1: `books` array item (sub-document)

Field	Type	Required	Constraints
`bookId`	ObjectId, ref `"books"`	Yes	—
`title`	String	Yes	—
`quantity`	Number	Yes	`min: 1`
`price`	Number	Yes	`min: 0`

Field 2: `shippingAddress` (sub-document)

Inline object (not an array) with all fields optional at the schema level: `fullName`, `street`, `city`, `state`, `postalCode`, `country`, `phone`, `instructions` — all `String`.

> Validation of required fields for delivery orders is handled by Zod refinement, not by Mongoose.

Field 3: `statusHistory` array item (sub-document)

Field	Type	Default
`status`	String	—
`changedAt`	Date	`Date.now`
`changedBy`	ObjectId, ref `"users"`	—
`note`	String, `maxlength: 500`	—

Main schema fields

Field	Type	Required	Default	Constraints
`userId`	ObjectId, ref `"users"`	Yes	—	—
`books`	Array (see above)	Yes	—	—
`subtotal`	Number	Yes	—	`min: 0`
`shippingFee`	Number	No	`0`	`min: 0`
`totalPrice`	Number	Yes	—	`min: 0`
`deliveryMethod`	String enum `["pickup", "delivery"]`	Yes	`"pickup"`	—
`shippingAddress`	Object (see above)	No	—	—
`paymentMethod`	String enum `["cash_on_delivery"]`	Yes	`"cash_on_delivery"`	—
`paymentStatus`	String enum `["pending", "paid", "failed", "refunded"]`	No	`"pending"`	—
`status`	String enum (OrderStatus values)	No	`"pending"`	—
`orderNotes`	String	No	—	`maxlength: 500`
`trackingNumber`	String	No	—	—
`cancellationReason`	String	No	—	—
`cancelledAt`	Date	No	—	—
`statusHistory`	Array (see above)	No	—	—

Schema options

Option	Value
`timestamps`	`true`
`toJSON`	`{ versionKey: false }`
`toObject`	`{ versionKey: false }`

> No `virtuals: true` needed — this model has no virtual fields.

> No custom indexes — `userId` and `status` queries are handled well enough by MongoDB's default behavior for this project.

Export

`model<IOrder>("orders", orderSchema)` typed as `Model<IOrder>`, exported as default `orderModel`

22.4. Order Validation

File: `src/validation/order.ts`

1. `shippingAddressSchema` (internal, not exported)

Field	Rules
`fullName`	Optional string
`street`	String, min 1
`city`	String, min 1
`state`	Optional string
`postalCode`	String, min 1
`country`	String, min 1
`phone`	String, min 1
`instructions`	Optional string

---

2. `cartSchema` — body of `POST /orders`

The main schema for order creation from the client.

Field	Rules
`books`	Array of `{ bookId: mongoDbIdSchema, quantity: number min 1 }`, min 1 item
`deliveryMethod`	Enum: `"pickup"` or `"delivery"`
`shippingAddress`	Optional, uses `shippingAddressSchema`
`paymentMethod`	Optional enum: `"cash_on_delivery"`
`orderNotes`	Optional string, max 500

Refinement: If `deliveryMethod === "delivery"` → `shippingAddress` must be present. Error path: `["shippingAddress"]`.

---

3. `updateOrderStatusSchema` — body of `PUT /orders/:id/status`

Field	Rules
`status`	Required enum (all OrderStatus values)
`note`	Optional string, max 500

---

4. `cancelOrderSchema` — body of `POST /orders/:id/cancel`

Field	Rules
`reason`	Optional string, max 500

---

5. `orderQuerySchema` — query of `GET /orders` (admin)

Extends `paginationSchema` using `.and()`:

```
export const orderQuerySchema = paginationSchema.and(z.object({ ... }))
```

Additional fields:

Field	Rules
`status`	Optional enum (all OrderStatus values)
`sortBy`	Optional enum: `"createdAt"`, `"totalPrice"`, `"status"`, `"updatedAt"` — default `"createdAt"`
`sortOrder`	Optional enum: `"asc"` or `"desc"` — default `"desc"` (kept as string, not transformed to 1/-1)
`userId`	Optional `mongoDbIdSchema`

> `sortOrder` stays as a string here — the handler converts it inline: `sortOrder === "asc" ? 1 : -1`.

---

6. `orderSchema` (full admin schema — not used in routes directly)

Define this for completeness — it validates the full order shape. Fields match `IOrder`: userId, books array, subtotal, shippingFee, totalPrice, deliveryMethod, shippingAddress?, paymentMethod, paymentStatus, status, orderNotes?, trackingNumber?.

---

7. Type exports

Type	From schema
`ShippingAddressInput`	`shippingAddressSchema`
`CartInput`	`cartSchema`
`OrderInput`	`orderSchema`
`UpdateOrderStatusInput`	`updateOrderStatusSchema`
`CancelOrderInput`	`cancelOrderSchema`
`OrderQueryInput`	`orderQuerySchema`

22.5. Order Handlers

File: `src/handlers/orders.ts`

Handler 1 : `createOrder` — `POST /orders` — Authenticated

This handler uses a transaction. The session must always be ended in a `finally` block.

1. Start session: `const session = await mongoose.startSession()` then `session.startTransaction()`
2. Get `userId` from `req.user._id` — destructure `books` (cartItems), `shippingAddress`, `deliveryMethod` (default `"pickup"`), `paymentMethod` (default `"cash_on_delivery"`), `orderNotes` from `req.body`
3. If `cartItems` is empty → return `400` "Order must contain at least one item"
4. Loop over each cart item — for each:
   - Fetch book: `bookModel.findById(item.bookId).session(session)` — if not found → abort transaction and return `404`
   - If `book.availableStock < item.quantity` → abort transaction and return `400` `Not enough stock for "${book.title}"`
   - Push to `orderBooks` array: `{ bookId: book._id, title, quantity, price: book.price?.current || 0 }`
   - Add `price × quantity` to `subtotal`
   - Decrement stock: `findByIdAndUpdate(item.bookId, { $inc: { availableStock: -item.quantity } }, { session })`
5. Calculate `shippingFee`: if `deliveryMethod === "delivery"` AND `subtotal < 50` → `5`, otherwise `0`
6. `totalPrice = subtotal + shippingFee`
7. Create order: `const [order] = await orderModel.create([{ userId, books: orderBooks, shippingAddress, deliveryMethod, paymentMethod, subtotal, shippingFee, totalPrice, status: "pending", orderNotes, statusHistory: [{ status: "pending", changedAt: new Date(), changedBy: userId }] }], { session })`
8. `await session.commitTransaction()`
9. Populate `books.bookId` (title, cover, author) and `userId` (firstName, lastName, email)
10. Return `201`
11. In `catch`: `await session.abortTransaction()` — return `500`
12. In `finally`: `session.endSession()`

> **Why `.create([...], { session })` with array syntax?** When passing a session option to `Model.create()`, Mongoose requires the data wrapped in an array.

Handler 2: `getOrders` — `GET /orders` — Admin

Reads from `req.parsedQuery`.

1. Destructure `limit`, `page`, `sortBy`, `sortOrder`, `status`, `userId` from `parsedQuery`
2. Build filter — add `status` and `userId` only if provided
3. Run in parallel: `find(filter)` with `.populate("userId", "firstName lastName email").populate("books.bookId", "title cover price")` + pagination + `.sort({ [sortBy]: sortOrder === "asc" ? 1 : -1 })` + `.lean()`, and `countDocuments(filter)`
4. Return `200` with `data` and `pagination`

Handler 3: `getUserOrders` — `GET /orders/my` — Authenticated

Reads directly from `req.query` (no query validation middleware on this route) — parse manually.

1. Get `userId` from `req.user._id`
2. Read `status`, `page` (default `"1"`), `limit` (default `"10"`) from `req.query`
3. Parse: `pageNum = Math.max(1, parseInt(page) || 1)` and `limitNum = Math.min(100, Math.max(1, parseInt(limit) || 10))`
4. Build filter `{ userId }` — add `status` if provided
5. Run in parallel: find with `.populate("books.bookId", "title cover price author")` sorted by `createdAt: -1` + skip/limit + `.lean()`, and `countDocuments`
6. Return `200` with `data` and `pagination`

Handler 4: `getOrderById` — `GET /orders/:id` — Authenticated

1. Get `id` from params, `userId` from `req.user._id`, determine `isAdmin` via `req.user.role === "admin"`
2. Find order with `.populate("books.bookId", "title cover author price").populate("userId", "firstName lastName email")`
3. If not found → return `404`
4. Extract userId from populated document: `(order.userId as { _id?: unknown })._id || order.userId`
5. If not admin and IDs don't match → return `403` "Access denied"
6. Return `200`

Handler 5: `updateOrderStatus` — `PUT /orders/:id/status` — Admin

1. Get `id` from params, `status` and `note` from `req.body`, `adminId` from `req.user._id`
2. Find order — if not found return `404`
3. Validate transition using this map:

Current status	Allowed next statuses
`pending`	`processing`, `cancelled`
`processing`	`shipped`, `cancelled`
`shipped`	`delivered`
`delivered`	`completed`
`cancelled`	none
`completed`	none

4. If transition invalid → return `400` `Cannot change status from ${currentStatus} to ${status}`
5. Set `order.status = status` — push to `order.statusHistory`: `{ status, changedAt: new Date(), changedBy: adminId, note }`
6. If `status === "cancelled"` → loop over `order.books` and `findByIdAndUpdate(item.bookId, { $inc: { availableStock: item.quantity } })` for each. Also set `order.cancellationReason = "Cancelled by admin"` and `order.cancelledAt = new Date()`
7. `await order.save()` — return `200`

Handler 6: `cancelOrder` — `POST /orders/:id/cancel` — Authenticated (owner only)

1. Get `id` from params, `userId` from `req.user._id`, `reason` from `req.body`
2. Find order — if not found return `404`
3. If `order.userId.toString() !== userId.toString()` → return `403` "You can only cancel your own orders"
4. If `order.status` not in `["pending", "processing"]` → return `400` "Order cannot be cancelled at this stage"
5. Loop over `order.books` → `findByIdAndUpdate(item.bookId, { $inc: { availableStock: item.quantity } })` for each
6. Set `order.status = "cancelled"`, `order.cancellationReason = reason || "Cancelled by user"`, `order.cancelledAt = new Date()`
7. Push to `order.statusHistory`: `{ status: "cancelled", changedAt: new Date(), changedBy: userId }`
8. `await order.save()` — return `200`

Handler 7: `processPayment` — `POST /orders/:id/pay` — Authenticated (owner only)

Mock handler — simulates a payment gateway confirmation.

1. Find order — if not found return `404`
2. If `order.userId.toString() !== userId.toString()` → return `403`
3. If `order.paymentStatus !== "pending"` → return `400` "Order payment is not pending"
4. Set `order.paymentStatus = "paid"`, `order.status = "processing"`
5. Push to `order.statusHistory`: `{ status: "processing", changedAt: new Date(), changedBy: userId }`
6. `await order.save()` — return `200`

Handler 8: `getOrderStats` — `GET /orders/stats` — Admin

Run all queries in parallel with `Promise.all`:

1. `countDocuments()` — total orders
2. `countDocuments({ status: "pending" })`
3. `countDocuments({ status: { $in: ["delivered", "completed"] } })`
4. `countDocuments({ status: "cancelled" })`
5. Aggregation: `$match: { paymentStatus: "paid" }` → `$group: { _id: null, total: { $sum: "$totalPrice" } }` — extract `result[0]?.total || 0`

Return `200` with `{ totalOrders, pendingOrders, completedOrders, cancelledOrders, totalRevenue }`

22.6. Order Routes

File: `src/routers/orders.ts` | **Prefix:** `/orders`

Register in this exact order — all fixed-path routes before `/:id`.

#	Method	Path	Middleware	Handler
1	GET	`/my`	`CheckAuth`	`getUserOrders`
2	POST	`/`	`CheckAuth`, `validateBodySchema(cartSchema)`	`createOrder`
3	GET	`/`	`CheckAuth`, `isAdmin`, `validateQuerySchema(orderQuerySchema)`	`getOrders`
4	GET	`/stats`	`CheckAuth`, `isAdmin`	`getOrderStats`
5	PUT	`/:id/status`	`CheckAuth`, `isAdmin`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(updateOrderStatusSchema)`	`updateOrderStatus`
6	GET	`/:id`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`	`getOrderById`
7	POST	`/:id/cancel`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`, `validateBodySchema(cancelOrderSchema)`	`cancelOrder`
8	POST	`/:id/pay`	`CheckAuth`, `validateParamsSchema(idParamsSchema)`	`processPayment`

> `GET /stats` must be before `GET /:id` — otherwise Express treats “stats” as a MongoDB ObjectId.

> `POST /:id/cancel` uses `cancelOrderSchema` (validates optional `reason` in body).

Mount in `app.ts`: `app.use("/orders", orderRouter)`